The Machine Speed Papers

Upstream

Where Human-in-the-Loop Goes at Machine Speed

Jonathan M. Watson
Independent Researcher
jonmwatson@gmail.com

The Machine Speed Papers, No. 4 — Version 1.0 — August 2026

Contents

Abstract 3
Chapter 1 — The Corpus Never Finishes 3
Chapter 2 — The Parties Already Asking 6
Chapter 3 — This Has Been Attempted Before 10
Chapter 4 — Most of It Is Not in the Policy Binder 13
Chapter 5 — What a Rule Has to Become 16
Chapter 6 — The Assessment 20
Chapter 7 — What Happens to Ten Thousand Policies 23
Chapter 8 — Four Weeks and Four Milliseconds 26
Chapter 9 — Both Halves, Permanently 30
Chapter 10 — The Morning the Table Went Down 35
Chapter 11 — The People Who Move Upstream 40
Chapter 12 — What You Actually Bought 44
Chapter 13 — The New Compliance Officer 46
Chapter 14 — Where the Value Actually Sits 49
Chapter 15 — What This Market Looks Like 52
Chapter 16 — What the Work Actually Looks Like 55
Chapter 17 — The Objections 58
Chapter 18 — What Is Still Missing 61

Abstract

Software now takes consequential action on its own. It places orders, moves money, approves claims, and grants access, and it does all of it faster than anyone can review. Governance that cannot be present at the moment of action has to be settled beforehand, and written so a machine can apply it without asking. This paper examines what happens when organisations try to do that at scale.

The usual conclusion is that the human leaves the loop. That conclusion is wrong. The human does not leave — the loop moves. Judgment that once sat at the moment of action, exercised case by case and mostly unrecorded, relocates to the moment the rule is settled: earlier, on the record, attributed, and applied to every case that follows. Human-in-the-loop does not weaken under machine-speed execution. It becomes more consequential, exercised by fewer people, at higher altitude, with their names on the result.

Which raises the question of how much of it there is to do, and the answer is that the work does not end. Governance is produced everywhere at once — by the business, by its regulators, by its contracts — while translation happens in one place, at the speed of the few people with the authority to settle what a rule actually means. Production outruns translation structurally, not temporarily. Every organisation running executable governance therefore operates in two regions at all times: a Compiled Region, enforced by machine, and an Uncompiled Region, still held by people. The boundary between them is the Uncompiled Frontier. It moves. It does not close.

Four structural consequences follow. Human judgment does not shrink; it migrates upstream, into designing constraints and deciding what cannot be compiled at all. Translation becomes a standing function rather than a project, and therefore a professional services market with no natural end date. The failures of compilation — ambiguity, contested authority, dead policy, undocumented exceptions — are not waste but the principal output, because each one is a question the institution had never been forced to answer. And enforcement architectures have to be built for a world where most governance never reaches them.

This is an opportunity rather than a burden. The institutions that build translation capacity early can launch products their competitors cannot govern, and the knowledge the work depends on is held by people who will not be there indefinitely. Each proposition here is independently testable. The paper provides a test for whether a policy can be compiled at all, a treatment of what should happen when governance is unavailable, and an account of the work, the roles, and the market the transition creates.

Chapter 1 — The Corpus Never Finishes

Software has begun to act. Not to recommend, not to prepare, not to route work to a person who decides — to act. It places orders with suppliers. It moves money between accounts. It approves claims, adjusts pricing, files documents, grants access, and commits the organisation to obligations it will be held to. The action completes in milliseconds and is not reversible in any practical sense once it has.

Every governance mechanism an organisation owns was built on an assumption this breaks. Approval workflows, delegated authority matrices, sign-off thresholds, four-eyes checks, escalation paths, audit review — all of them assume a pause. Somewhere between intention and consequence there is an interval in which a person can look at the thing, apply judgment, and say yes or no. The entire apparatus of corporate governance is constructed inside that interval.

When execution takes milliseconds, the interval is gone.[1] Not shortened — gone. There is no arrangement of committees, on-call rotations, or approval tooling that puts human judgment inside a window measured in thousandths of a second. This is a physical limit, and it does not yield to process improvement or additional headcount.

What remains is a choice with only two options.

The first is to let the system act and review afterwards. This is what most organisations are doing now, and it produces governance that describes rather than controls. The action already happened. The review establishes what occurred and who might be answerable, which is useful for reporting and useless for prevention. When the consequence is material — a payment that should not have been made, a claim that should not have been approved, an access grant that should not have been issued — after-the-fact review arrives to find the money gone, the commitment binding, and the exposure already carried.

The second is to decide in advance. If judgment cannot be present at the moment of action, then the answer must already exist at that moment, in a form the system can apply by itself, without asking anyone. The rule must be resolved before the agent runs, expressed precisely enough that the system can evaluate it against a proposed action and reach a determination in the time available.

That transformation — from governance written for a person to read to governance a system can enforce unaided — is policy compilation. It is not a technology preference or an efficiency measure. It is the only mechanism by which an organisation's rules reach an autonomous action at all. Everything else arrives too late.

The pressure to do it is not internal. Insurers underwriting autonomous execution are asking what was enforced and what proof exists. Regulators across several jurisdictions have moved from asking whether an organisation has a policy to asking whether it can demonstrate the policy was applied. Counterparties are beginning to ask the same question in procurement. In each case the answer requires that governance was operative at the moment of execution, which requires that it was compiled beforehand.

So organisations begin. And the first question every one of them asks is how long it will take.

The question assumes an answer exists — a corpus of finite size, a rate of translation, and a date on which the work is done. Programmes are scoped this way, budgets approved this way, and vendors selected on their claimed throughput.

[1] The latency constraint on human intervention in automated execution is established in the functional safety literature, where safety-related systems are specified to act within bounded intervals that exclude operator response. See IEC 61508 (2010), *Functional Safety of Electrical/Electronic/Programmable Electronic Safety-related Systems*, International Electrotechnical Commission, Geneva.

The assumption is wrong, and the remainder of this paper depends on understanding why.

An institution's governance is not a fixed body of text awaiting conversion. It is a corpus under continuous production. New legislation arrives. New jurisdictions are entered. New products create obligations that did not exist when the last policy was written. Incidents produce controls. Reorganisations redistribute authority without documenting the redistribution. Contracts impose terms that bind operations more tightly than internal policy does. Beneath all of it sits an exception layer that forms continuously in daily operation and is never written down — the accumulated practice of people deciding what to do when the rule does not fit, which functions as governance whether or not anyone has recorded it.[2]

That production is distributed. It happens across every function, every geography, and every regulator with authority over the institution, simultaneously and without coordination.

Translation is not distributed. It is concentrated, and it is rate-limited by something that does not respond to tooling.

Converting a well-specified rule into executable form is mechanical work, and increasingly automated work. That is not where the time goes. The time goes into what has to happen first. Someone has to say who is actually allowed to approve this. Someone has to write down what "excessive" means, for the first time. And someone has to decide what the system does in a situation the policy never imagined.

The translation team cannot do any of that on its own. It needs people with the standing to decide, making calls that have never been made before — because until now, nothing forced anyone to make them.

Those people are few, they hold other responsibilities, and their availability is the binding constraint on the entire exercise.

The result is a mismatch that does not resolve. Governance gets made everywhere at once. It gets translated in one place, slowly. That gap does not close, and calling it a backlog misdescribes it — backlogs are cleared by capacity, and this one is not. Partial compilation is not a stage. It is the permanent operating condition.

Every institution running autonomous systems under executable governance will, at all times, operate across two regions. In the Compiled Region, constraints are deterministic, enforcement occurs at machine speed, and authority is resolved in advance. In The Uncompiled Region, everything remains as it was: ambiguous, contextual, interpreted by people at human speed, with authority sometimes contested and often undocumented.

[2]The divergence between documented procedure and operational practice is well established in safety science, where it is framed as the gap between work-as-imagined and work-as-done. Formal descriptions of work systematically omit the adjustments practitioners make to absorb variability, and those adjustments function as the operative rules of the system regardless of whether they appear in the procedure. See Erik Hollnagel, *Safety-I and Safety-II: The Past and Future of Safety Management* (Farnham: Routledge, 2014).

The boundary between them is the Uncompiled Frontier. It moves as translation proceeds and as new governance is produced, and its position at any moment is an empirical fact about the institution. It does not close. A frontier is not a queue.

Four structural consequences follow, and the rest of this paper derives them.

Human judgment does not leave. It moves — into the region that cannot be compiled, and into the prior decision about what gets compiled at all.

Translation becomes a standing function rather than a project. It is permanent, it is interpretive, and it requires people with the authority to settle questions nobody has settled before. That combination makes it a professional services market, the same way financial control and data protection became professional services markets.

The failures of compilation are the output, not the waste. Every ambiguity found, every contested authority surfaced, every dead policy identified is a question the institution had never been forced to answer.

And anyone building enforcement has to design for a world where most of an institution's governance never arrives. That changes what the enforcement layer is for.

The claim is falsifiable, and the test is simple. Find an institution of any real size that has compiled its whole governance corpus and held that state through one normal year — a regulatory change, a product launch, a reorganisation.

The argument says you will not find one. It also says that any organisation which believes it has got there has counted wrong. Usually they counted the written policies and left out the delegated authority, the contracts, and the exceptions nobody wrote down. That is normally the bigger half.

The pressure to begin does not come from inside the organisation, and it is not an argument anyone gets to win. Three parties are already asking the question that compels it.

Chapter 2 — The Parties Already Asking

The pressure to compile governance does not originate inside the organisation. It arrives from three parties who have their own reasons for asking, and it arrives whether or not the organisation has an opinion about autonomous execution. The insurers moved first

In January 2026, the Insurance Services Office — the standard-forms arm of Verisk, whose language the commercial market builds on — issued endorsements permitting carriers to exclude generative AI losses from commercial general liability policies.[3] Three forms were published. The broadest strips bodily injury, property damage, and personal and advertising injury arising from generative AI; a second narrows

[3] Insurance Services Office (Verisk Analytics), endorsements CG 40 47, CG 40 48, and CG 35 08, January 2026 edition. CG 40 47 excludes bodily injury, property damage, and personal and advertising injury arising out of generative artificial intelligence under Coverage A and Coverage B; CG 40 48 applies to Coverage B alone; CG 35 08 extends the exclusion to the products and completed operations coverage part. Optional attachment at the discretion of the issuing carrier.

to personal and advertising injury; a third extends the exclusion to products and completed operations. They carry a January 2026 edition date and begin attaching at renewal.

Verisk has since confirmed it is evaluating further options addressing agentic AI specifically.[4]

Cyber carriers have moved in parallel, introducing AI sublimits rather than outright exclusions. And a portion of the narrowing is not visible in endorsement language at all — carriers are managing the same exposure through revised forms, narrower interpretation, and underwriting practice, which means a policyholder can lose coverage without ever being handed a document that says so.[5]

The reasons carriers give are specific and worth reading closely, because they define what will resolve them. Underwriters cite the opacity of these systems and the difficulty of modelling loss scenarios.[6] Neither is a complaint about AI being dangerous. Both are complaints about it being unpriceable.

Underwriters also distinguish generative from agentic exposure, and regard the second as materially more severe — generative systems produce outputs, agentic systems reason and decide, and the risk profile changes accordingly.[7] Every organisation now considering autonomous execution is moving toward the exposure the market has already identified as the harder one.

This has a precedent, and the precedent is the reason the argument matters.

Cyber risk followed exactly this sequence.[8] It sat silent inside general liability policies for years, uncosted and unexcluded. Carriers eventually recognised the exposure, wrote exclusions, and a standalone market formed to cover what the exclusions removed. Then the decisive step: standalone cyber underwriters, unable to price an unmodellable risk, began requiring specific controls as a condition of coverage.[9] Multi-

[4]Reported in *The Insurer*, "Verisk weighs new exclusions for agentic AI risks," 10 July 2026. Verisk confirmed it is assessing additional options addressing AI-related exposures including agentic AI, and stated that any such language would be optional, with adoption at the discretion of each insurer according to its underwriting guidelines and risk appetite.

[5]Fenwick, "The End of 'Silent AI'? Emerging AI Exclusions, Coverage Fragmentation, and Practical Implications for Policyholders," June 2026. Carriers are managing AI exposure not only through new exclusions but through existing policy limitations, narrowing interpretations, revised coverage forms, and underwriting practice — producing coverage erosion that is not always transparent to the policyholder.

[6]Fenwick, ibid. Cited underwriting concerns include the opacity of AI systems, the difficulty of modelling loss scenarios, and the potential for systemic or large-scale harm.

[7]*The Insurer*, "Standalone AI liability market takes shape with underwriting discipline key to MGA success," 24 April 2026. George Lewin-Smith, co-founder and CEO of Testudo, distinguishes generative AI as concerned with system outputs from agentic AI, which combines generative capability with reasoning and decision-making, and describes the agentic risk profile as higher.

[8]Fenwick, ibid. The AI coverage transition is characterised as mirroring the evolution of cyber risk, which was initially carried as silent exposure within traditional policies before insurers introduced exclusions, affirmative endorsements, and standalone products — with AI following a comparable path on a compressed timeline and with a more fragmented market response.

[9]Howden, *Cyber Insurance: A Hard Reset* (London: Howden Broking Group, 2022). Following the loss experience of 2020–2021, standalone cyber underwriters moved from recommending security controls to requiring them as a condition of coverage, with multi-factor authentication, endpoint detection and response, and tested offline backups functioning as preconditions to placement rather than as pricing factors.

factor authentication. Endpoint detection. Tested backups. Not recommendations — preconditions. An organisation without them could not buy the policy at any price it was willing to pay.

AI is following that sequence on a compressed timeline. The exclusions have been written. The standalone market is forming — Testudo, a Lloyd's-backed managing general agent, began underwriting US mid-market AI liability in early 2026, and it is not alone.[10]

The step that has not yet happened is the control requirement. Nothing in the current evidence shows underwriters mandating demonstrable enforcement at the execution boundary. But everything that produced it in cyber is already here. Carriers have cut the exposure out of general coverage. A standalone market has formed and has to price what is left. And the obstacle those underwriters name — they cannot see inside, they cannot model the loss — is exactly the one that demonstrable control removes.

The practical advice already circulating to risk managers is the tell. Audit your policies before renewal, identify where AI is used, and document your governance so it can be presented to the underwriter.[11] That is the market instructing organisations to prepare an argument. Compiled governance is the only form of that argument that survives scrutiny, because it is the only form that describes what the system could not do rather than what the organisation intended.

The regulators have already decided this once. In one domain, the question is settled, and it was settled sixteen years ago. When execution in equity markets moved past human reaction time, regulators did not ask firms to supervise more attentively.[12] They required controls that operate before the order reaches the market — pre-trade checks on size, position, credit, and regulatory compliance, applied automatically, at machine speed, without a person in the interval. Governance was compelled into executable form because nothing else could reach the point of consequence.

That is policy compilation, mandated, in production, in the highest-consequence financial environment that exists. It is not a proposal. It is how the market has operated since 2010.

The broader regulatory movement runs the same direction more slowly. Across jurisdictions, the question is shifting from whether an organisation holds a policy to whether it can demonstrate the policy applied to a specific action. The second question cannot be answered by documentation. It can only be answered by a system that evaluated the action against the policy at the moment it was proposed and produced a record of having done so.

[10] *The Insurer*, 24 April 2026. Testudo, a Lloyd's-backed AI liability managing general agent, commenced underwriting US mid-market enterprises in early 2026.

[11] Fenwick, ibid. Guidance to policyholders includes auditing existing coverage in advance of renewal, identifying where AI systems are deployed across the organisation, and documenting AI governance in a form that can be presented to underwriters.

[12] SEC Rule 15c3-5 (2010), Market Access Rule; SEC Release No. 34-63241 (2010), *Risk Management Controls for Brokers or Dealers with Market Access*, U.S. Securities and Exchange Commission. The rule mandates pre-trade controls covering order size, position limits, credit limits, and regulatory compliance, applied automatically prior to order entry.

Where autonomous systems now handle money, regulators are also confronting the identity question underneath — whether a machine acting under delegated authority requires verification standards of its own, linked to the legal entity behind it.[13] Authority that must be verified is authority that must first be declared, and declared in a form a system can check.

The counterparties will ask last and hardest. The third party has no regulatory mandate and no underwriting model. It simply refuses to sign. This is the SOC 2 pattern, and the software bill of materials pattern after it. Neither was imposed on the commercial market by an authority. Both became near-universal because buyers with many vendors grew tired of evaluating trust one relationship at a time and converged on a common artifact that let a single checklist replace fifty bespoke reviews.[14]

An organisation whose autonomous systems act on a counterparty's data, funds, or obligations will be asked what governs those systems. The current answer — a policy document and an assurance — is the same answer that stopped working for security posture a decade ago, for the same reason: it is unverifiable, and the buyer has no way to distinguish a good answer from a confident one.

Procurement moves slowly and then all at once, and it moves without appeal. There is no negotiating with a requirement that has become standard.

What all three are asking for The three parties have different motives. The insurer wants to price. The regulator wants a rule that actually bites. The counterparty wants to stop evaluating trust individually. None is interested in governance philosophy.

But the question underneath is identical, and it is not *do you have a policy*. It is *what did your system prevent, and how do you know*.

That question has one honest answer. Someone decided the answer in advance. The rule was written so a machine could apply it without asking. The machine checked it the instant the action was proposed. And it kept the receipt. Everything upstream of that — the policy binder, the committee, the training, the attestation — describes an intention. Only compiled governance describes a constraint.

Which raises the obvious question. If compilation is the required response, and the demand is arriving from three directions at once, why is this not already a solved discipline with established practice and mature tooling?

It has been attempted before. Understanding why those attempts stalled is the precondition for claiming that this one is different.

[13] International Monetary Fund, "How Agentic AI Will Reshape Payments," IMF Notes Vol. 2026, Issue 004, April 2026. The note observes that agentic AI requires regulators to consider a shift from Know-Your-Customer to Know-Your-Agent requirements, with mandated verifiable identities for financial agents linked to legal entities.

[14] On SOC 2: American Institute of Certified Public Accountants, *Trust Services Criteria* (New York: AICPA, 2017). On software bills of materials: Executive Order 14028, *Improving the Nation's Cybersecurity*, 12 May 2021, and National Telecommunications and Information Administration, *The Minimum Elements for a Software Bill of Materials*, July 2021. In both cases adoption across the commercial market was driven principally by buyer demand rather than by regulatory mandate, with the artifact becoming a standard procurement precondition.

Chapter 3 — This Has Been Attempted Before

The idea that an organisation's rules could be extracted, formalised, and executed by machine is not new. It is roughly as old as commercial computing, and it has been attempted in at least four distinct waves, each with substantial investment, credible technology, and serious people behind it.

None of them produced a durable profession. Understanding why is the precondition for claiming that this time is different — and for identifying which of the old failure modes are still available.

Four waves **Expert systems.** In the 1980s, knowledge engineering set out to capture the reasoning of human specialists in machine-executable form. The inference engines worked. What defeated the field was named at the time and never solved: the knowledge acquisition bottleneck.[15] Extracting what an expert knew, in a form precise enough to encode, proved far harder and slower than executing it once encoded. The technology was not the constraint. The expert's availability was.

Business rules management. From the mid-1990s, a substantial commercial category formed around separating business logic from application code. Rules would live in a repository, be authored by business users rather than programmers, and change without redeployment. The category was real — ILOG, Blaze Advisor, Pegasystems, Corticon, later Drools — and produced a methodology literature, a manifesto, and a named role: *the business rules analyst*.[16]

That role never established itself as a distinct profession in the way the manifesto anticipated. It was absorbed back into business analysis and solution architecture.[17] The repositories grew, accumulated conflicts nobody was accountable for resolving, and authoring drifted back to technical staff because business users could not express rules with the precision execution required.

Computational law. Beginning in the 2000s and accelerating academically, an effort to render legislation and regulation directly executable — LegalRuleML, Akoma Ntoso, Catala, and the work around Stanford's CodeX among others.[18] The results are real

[15]The knowledge acquisition bottleneck was identified by Edward Feigenbaum as the principal constraint on expert systems development. See Feigenbaum, E.A. (1977). The art of artificial intelligence: themes and case studies of knowledge engineering. *Proceedings of the 5th International Joint Conference on Artificial Intelligence*, vol. 2, 1014–1029. The problem is treated as the central practical difficulty of the field in Hayes-Roth, F., Waterman, D.A., & Lenat, D.B. (eds.) (1983). *Building Expert Systems*. Addison-Wesley.

[16]The Business Rules Manifesto (2003), Business Rules Group, ed. Ronald G. Ross, sets out the separation of business rules from application logic as an explicit principle. See also Ross, R.G. (2003). *Principles of the Business Rule Approach*. Addison-Wesley; and von Halle, B. (2001). *Business Rules Applied*. Wiley.

[17]The business rules analyst was defined as a distinct role in the methodology literature of the period; see von Halle (2001) and Ross (2003), above. No comparable body of certification, professional association, or standing job classification developed around it, and the responsibilities were absorbed into business analysis and solution architecture.

[18]Athan, T., Boley, H., Governatori, G., Palmirani, M., Paschke, A., & Wyner, A. (2013). OASIS LegalRuleML. *Proceedings of the 14th International Conference on Artificial Intelligence and Law*,, 3–12. Akoma Ntoso Version 1.0 was approved as an OASIS Standard in August 2018, covering parliamentary, legislative, and judicial documents. On Catala, see Merigoux, D., Chataing, N., & Protzenko, J. (2021). Catala: A Programming Language for the Law. *Proceedings of the ACM on Programming Languages*, *5*(*ICFP*), *Article 77.*

and in some cases in production, but confined to narrow, unusually well-specified domains: tax computation, benefits eligibility, statutory calculation. The approach has not generalised to institutional governance, and its practitioners have not claimed it has.

Policy-as-code. The current wave, and the successful one. Open Policy Agent and Rego, Open Policy Agent and Rego, Cedar, Sentinel, Kyverno, alongside decision standards like DMN and control-catalogue formats like OSCAL.[19] These systems work, are widely deployed, and compile policy into enforcement that runs at the point of decision.

OSCAL deserves separate notice, because it attempts something closer to the problem set out here than the others do. It does not compile a single decision at a service boundary; it renders an entire control corpus machine-readable, along with the assertions an organisation makes about implementing it. It is also the clearest illustration of the limit. OSCAL represents controls, profiles, and claims of implementation with precision. It does not resolve what a control means for a specific proposed action, and it does not settle who holds authority to decide when the control is ambiguous. It formalises the record of governance rather than the determination. That distinction recurs throughout this paper.

The fourth wave's success is the most instructive thing in this chapter, because it succeeded under conditions the earlier three lacked.

Why the first three stalled Three structural conditions were absent, and their absence explains the pattern. Beneath all three sits a shared premise that none of the four waves stated and none of them questioned. Every one of them treated the governance corpus as a body of finite extent — something that could in principle be captured, encoded, and completed. The programmes differed in method and in target, but each assumed a terminus. That assumption is what this paper rejects, and rejecting it changes what the work is for.

There was no fixed compilation target. A business rules engine could express nearly anything. That flexibility, presented as a virtue, made the work unbounded — there was no definition of what it meant for a rule to be finished, and therefore no definition of when a programme was complete. Scope expanded until the budget was exhausted. Policy-as-code inverted this: the target is narrow and specified, an authorisation determination at a service boundary, and a rule is either expressible in that form or it is not.

Nothing was waiting for the output. Compiled rules fed systems that still had a human in the loop, able to notice an error and correct it. Compilation fidelity therefore did not have to be high, because the consequences of imprecision were absorbed downstream. Without pressure for precision, precision was not achieved. Policy-as-code emerged

[19] Open Policy Agent is a CNCF graduated project; its policy language is Rego. Cedar is an open-source authorisation policy language released by Amazon Web Services. Kyverno is a CNCF policy engine for Kubernetes. Decision Model and Notation (DMN) is an Object Management Group standard for decision logic. The Open Security Controls Assessment Language (OSCAL) is a NIST-maintained set of machine-readable formats for security control catalogues, profiles, implementation statements, and assessment results.

inside infrastructure where no such absorption exists — an admission controller must return a decision, and there is no one behind it.

Nothing external compelled it. No regulator required it, no insurer priced it, no counterparty demanded it. The work was therefore always deferrable in favour of something with a deadline, and it was consistently deferred. Policy-as-code arrived attached to cloud infrastructure that could not operate without it.

Chapters 1 and 2 establish that all three conditions have now reversed for governance generally, not merely for infrastructure authorisation. Autonomous execution supplies the second. Insurers, regulators, and counterparties supply the third. The first — a fixed compilation target for institutional governance rather than service-boundary authorisation — is the subject of Chapter 5.

The condition that has not reversed One failure mode from the first wave is untouched, and it is the one that matters most.

The knowledge acquisition bottleneck was never a tooling problem. Every subsequent generation treated it as one, and every generation built better authoring environments, friendlier syntax, and eventually natural-language extraction — on the assumption that if capture were made easy enough, the bottleneck would clear.

It did not, because the bottleneck was never capture. It was that the knowledge in question frequently did not exist in a settled form to be captured. When an expert is asked to state a rule precisely, the request often produces not a transcription but a decision — one that has never been made, because operating practice never required it. That decision requires standing to make, agreement among people who have not previously had to agree, and time from individuals who have other responsibilities.

This is Chapter 1's finding restated in the language of the field that discovered it forty years ago. It is why the corpus never finishes, and it is why no advance in extraction technology will change that. The tooling compiles what has been decided. It cannot decide.

Any current vendor claiming to have solved policy compilation through better extraction has misidentified the bottleneck in exactly the way three prior waves did.[20]

What is actually new The claim here is narrow, and worth stating narrowly.

Machine-executable policy is old news. Rule engines, decision tables, policy-as-code, computational law — all of them turn human rules into running logic, and they have for decades. That is not the contribution.

What is new is the situation the work now happens in. Four things have changed at once.

[20]Natural-language-to-policy pipelines report high compile rates against benchmark datasets. The claim here is not that these systems fail at what they measure. It is that the measured quantity is extraction fidelity against policies already precise enough to serve as benchmarks, which is not the constraint identified in this chapter.

Execution no longer waits. There is no interval to review a decision in, so the rule has to be settled before the system runs.

The target is fixed. Compilation now means resolving a policy into a specific structure, not expressing it in whatever form a rule engine will accept. That gives the work a finish line it never had.

Someone outside is asking. Insurers, regulators, and buyers now compel the work. It can no longer be deferred in favour of whatever has a deadline.

And the corpus never empties. The work does not end, so the capability cannot be temporary.

Each of those conditions has appeared before, in isolation. All four together have not. That is what turns a project into a function, and a function into a profession.

Chapter 4 — Most of It Is Not in the Policy Binder

Ask an organisation how big its governance corpus is and you will get a number. Four hundred policies. Twelve hundred. Whatever the policy management system reports.

That number is wrong, and not by a little. It is the smallest layer of a much larger pile, and it is the only layer anyone has counted, because it is the only layer that lives in one place with a search box.

Here is what is actually in the pile.

Written policy. The binder. Real governance, properly the first thing anyone looks at. Also the layer most likely to be out of date, because it is the layer with a formal review cycle that everybody defers.

Legislation and regulation. Not yours. You did not write it, you cannot change it, and it changes without asking you. A bank operating in four countries is governed by four sets of rules that were drafted by people who never spoke to each other. Some of what they require contradicts what the others require.

Contracts. This is the one that surprises people. Customer agreements, supplier terms, licensing conditions, data processing agreements. They bind the organisation as hard as any internal policy and often harder. A single enterprise customer can impose a restriction stricter than anything in your own handbook, and it applies only to that customer's data, and nobody outside the account team knows it exists.

Standards and certifications. SOC 2, ISO, sector schemes, whatever the industry body requires this year. Each one carries controls the organisation has committed to operating, and each commitment was made by someone who is no longer in the role.

Delegated authority. Who is allowed to approve what, and up to how much. This exists as a matrix somewhere. It is usually years old, frequently contradicted by how people actually work, and almost never reflects the last two reorganisations.

Risk appetite. Set at board level, in sentences like *we accept moderate operational risk in pursuit of growth.* True, agreed, minuted. Also completely disconnected from any

specific action anyone could take.[21] Somebody has to turn it into a number before it governs anything.

Procedures and operating models. Not what the rule is, but how the work gets done. The order of steps. What gets checked before what. Which system is the record. These are governance too, because deviating from them is how things go wrong.

And the exceptions. This is the largest layer, and it is not written down anywhere.

Every organisation runs on a body of accumulated practice about what to do when the rule does not fit. Approve it anyway if the customer is on the retention list. Skip the second signature if it is a renewal. Escalate to Maria, not to the person named in the procedure, because Maria actually knows. None of this is in a document. All of it is governance. It determines what happens.[22]

That layer is unbounded, and the reason is arithmetic.

Written policy is finite. Somebody has to draft it, approve it, and store it, and all three cost money, so the corpus stays countable. Four hundred policies. Twelve hundred. There is a number.

Exception practice has no such limit. Every policy generates edge cases, every edge case gets handled, and each handling becomes precedent for the next one. Nobody drafts it. Nobody approves it. Nobody stores it. And critically, nobody retires it — a written policy at least has a review date it can fail to meet, while an undocumented workaround simply persists until the person holding it leaves.

One layer is bounded by the cost of writing things down. The other is bounded by nothing at all, and has been accumulating for as long as the organisation has existed. Whether it is larger in any given institution is not something anyone can currently demonstrate. What can be shown is that only one of the two has a mechanism that limits its growth, and only one of the two has a mechanism that removes anything from it.

You cannot count it. That is the point. It is the largest part of the governance corpus and the only part with no inventory, no owner, and no way to search it.

This is why compilation is a services problem and not a software problem. If governance were the policy binder, a good extraction pipeline would eventually finish the job. But the biggest layer is not in the binder, is not in any system, and cannot be extracted from a document that does not exist. It only comes out of people — sitting with the claims team and asking what they actually do when the rule does not fit, then

[21] The gap between board-level risk appetite statements and operational limits is treated in the enterprise risk management literature and in supervisory guidance. See COSO (2017), *Enterprise Risk Management — Integrating with Strategy and Performance*, Committee of Sponsoring Organisations of the Treadway Commission; and the Financial Stability Board's *Principles for an Effective Risk Appetite Framework* (2013), which distinguishes qualitative appetite statements from the quantitative limits required to operationalise them.

[22] Hollnagel (2014), above. The divergence between documented procedure and operational practice is well established; what follows in the text is an argument about the relative size of the undocumented layer, which is derived from the asymmetry in how the two layers accumulate rather than from measurement. No inventory of the undocumented layer exists, for the reason the argument itself gives.

getting someone with authority to decide whether that practice becomes a rule or gets stopped.

No tool does that. People do that, at people speed, and there is a very large amount of it.

Two things happen when the inventory gets built, and both are painful.

The first is that the layers turn out to disagree. A contract requires something the internal policy prohibits. The delegated authority matrix says one thing and the approval system enforces another. Two regulators want incompatible treatments of the same record. These conflicts have existed the whole time. They were survivable because a human sat in the middle of every case and quietly resolved them, one at a time, without telling anyone.

Remove the human and the conflict has to be resolved once, in advance, in writing. That is a decision somebody now has to own.

The second is that a good deal of the pile turns out to be dead. Policies that reference systems long since retired. Controls written for a regulation since superseded. Procedures for a business the company exited. Nobody removed them because nobody was accountable for removal, and there was no cost to leaving them there.

There is a cost now. Dead policy in a compilation programme consumes exactly as much analyst time as live policy, right up until someone with standing confirms it is dead.

The obvious reaction to all of this is that the task is impossible. If the corpus is that large and most of it is not written down, nobody is ever going to compile it.

Correct. Nobody is. That was Chapter 1.

But it does not follow that the work is hopeless, because you never had to compile the corpus in the first place.

You compile the part of it that touches something a machine is allowed to do.

If an agent can approve claims up to a limit, then claims policy, the delegated authority behind it, the regulatory requirements on claims handling, and the contract terms of the affected policyholders all have to be compiled. Everything else in the pile stays exactly where it is, governing people, at human speed, as it always has. Travel expenses do not need compiling. The code of conduct does not need compiling. The dress code most certainly does not.

That is the scoping move, and it is the difference between a programme that finishes something and a programme that boils the ocean. The compilable surface is not defined by the size of the corpus. It is defined by what you have given your machines permission to do. This is also what fixes the position of the Uncompiled Frontier at any moment: not the ratio of translated policy to untranslated policy, but the relationship between what the machines are permitted to attempt and what has been settled about those attempts. Widening machine authority moves the Frontier outward faster than translation moves

it in. The two are not independent, and only one of them is under the institution's direct control.

Which means the first real decision is not a policy decision at all. It is deciding what the machine is allowed to attempt. Narrow that, and the compilation problem narrows with it. Widen it later, and you compile more.

So the question becomes precise. For a given set of actions a machine may take, what must a rule become before that machine can enforce it?

That has an exact answer, and it is shorter than anyone expects.

Chapter 5 — What a Rule Has to Become

The six requirements in this chapter are not proposed. They are derived, and they have been derived twice.

The first derivation came from a constitutional question — what is the minimum a governance system must contain to be legitimate at all. This chapter derives them again from an operational one: what does a machine physically need in order to apply a rule with no person present. Different starting points. Same six.

That is worth knowing, because it means the list is not a design preference. Nobody chose it. It is what the situation leaves behind, and anyone who works the problem carefully will land on the same set. The derivation below is self-contained — you do not need the earlier work to follow it, and you can check it yourself against any rule you like.

When a claims handler reads *approve routine claims up to the standard limit*, she does something that looks simple and is not. She looks at the claim in front of her — this one, this amount, this policyholder. She works out whether it counts as routine and whether it's under the limit. She checks, usually without noticing, whether she is the one allowed to approve it. She stops thinking and reaches an answer, because the claim has to go somewhere today. She approves it or she doesn't, and that's binding — she can't half-approve a claim. And if the limit is unclear or the system is down, she knows what to do instead, which is ask somebody.

Six things happened there. It took her maybe thirty seconds, and she could not have listed them afterwards.[23]

A machine has to be handed all six, separately, in advance. That is the whole of what compilation means, and this chapter derives the list rather than asserting it.

[23]The inability of experts to articulate the steps of their own routine judgments is well established in the expertise literature, and is a principal reason knowledge elicitation is slow. This is the mechanism underlying the knowledge acquisition bottleneck discussed in Chapter 3: the practitioner's competence is not stored in a form that can be recited.

Deriving it

Start with the only thing we know for certain: the machine is about to do something irreversible, and nobody is watching.

Something has to be on the table. A rule cannot be applied to nothing. Before any check can run, there has to be a specific proposed action — this payment, this amount, this recipient, now. Not a category of action. This one. If the system cannot say precisely what is being proposed, there is nothing to evaluate and the rest of the list is unreachable.

There has to be a rule it can actually check. Written policy is addressed to a reader who can interpret. A machine cannot interpret; it can only test. So the rule has to arrive as a condition the machine can evaluate against the proposal without judgment. *Above two hundred and fifty thousand* is such a condition. *Excessive* is not, until somebody says what number it means.

Somebody has to be allowed to do it. A condition being satisfied is not the same as the actor being permitted. The claim may be routine and within limit, and this particular AI agent may still have no business approving it. Authority is a separate question from correctness, and it has to be checked separately, against a record of who holds what. Most policies assume this record exists. It usually does not, or it is wrong.

The check has to finish. Whatever the machine does with these inputs, it has to stop. Not eventually — inside the window before the action commits. A rule that requires unbounded reasoning, or a lookup that may not return, cannot be enforced at the boundary no matter how well drafted it is. This is the requirement most often missed, because in human hands every rule terminates; the handler simply stops thinking and decides.[24]

Something has to come out, and it has to bind. The result cannot be advice. If the machine can proceed regardless of the outcome, no rule was enforced — a suggestion was logged. The output has to determine what happens next, and it has to be simple enough to leave no interpretive room. Yes or no. Anything richer — *maybe*, *escalate*, *proceed with caution* — pushes interpretation back into the moment where there is nobody to do it.[25]

And there has to be an answer for when this breaks. The authority record will be unreachable. The rule will not have been compiled yet. The evaluation will not return in time. These are not hypotheticals; they are Tuesday. Something happens in those moments whether or not anyone chose it. The only question is whether the behaviour was decided in advance by a person with the standing to decide it, or discovered afterwards by whoever is cleaning up.

[24] Bounded execution time is a foundational requirement in hard real-time systems, where a computation that produces a correct result after its deadline has failed. See Kopetz, H. (2011). *Real-Time Systems: Design Principles for Distributed Embedded Applications*, 2nd ed., Springer; and Buttazzo, G. (2011). *Hard Real-Time Computing Systems*, 3rd ed., Springer.

[25] The binary constraint is derived rather than surveyed. Any output value that does not itself determine what happens next requires interpretation, and interpretation requires either a person or a further evaluation cycle. At the execution boundary neither is available, which leaves two values. Richer output models are common in systems that retain a human reviewer downstream; the argument here applies only where none exists.

Call them what they are: the six questions. Every one has to be answerable before a machine can enforce a rule, and every one has to be answered by a person, in advance.

1. **What is being proposed:** The specific action, right now.
2. **What the rule actually says:** As a condition a machine can test, not a sentence a person interprets.
3. **Who is allowed:** Checked against a record of who holds what authority.
4. **Whether the check finishes:** Inside the window, every time.
5. **What comes out:** Yes or no, and it binds.
6. **What happens when it breaks:** Decided beforehand by someone with the standing to decide it.

That is six, and the list closes.[26]

Why six questions and not four, or ten

Because each question catches a failure none of the others would.

Drop the proposed action and there is nothing to check. Drop the testable rule and the machine has to interpret, which is the thing it cannot do. Drop authority and correct actions get taken by actors with no right to take them. Drop termination and the boundary hangs, which at machine speed is a failure, not a delay. Drop the binding output and you have built a logging system. Drop the failure behaviour and the system improvises during exactly the conditions where improvisation is most dangerous.

Nothing else is load-bearing. Logging matters, but it records the decision rather than making it. Notification matters, but it happens after. Escalation matters, and it is a form the binding output takes, not a seventh thing. Reputation, cost, sequencing, retry — all real, all downstream of a decision that has already been made.

Six is not a design choice. It is what remains when you take away everything a machine can do without.

What this gives you

A definition of *finished*. That sounds small. It is the thing every previous attempt at this lacked, and it is why they died.

Chapter 3 described four waves of people trying to make policy executable. Look at what they were compiling *into*. A rule engine will accept nearly any rule you can express. That was sold as the great advantage — no constraints, model anything, the tool will not stand in your way. Which means there was never a point at which a rule was done.

[26]The same minimal set was derived independently in earlier work in this series, from a constitutional starting point rather than an operational one, and arrived at the same six elements. See Watson, J.M., *Constitutional Compression: The Emergence of Minimal Governance at Machine Speed*. That two derivations from different premises converge on the same list is the basis for treating the set as a property of the situation rather than a design choice — a point the portability argument in Chapter 12 depends on.

You could always make it more precise. You could always add a condition, cover another case, handle another exception. Nothing in the tool ever said *stop, that one is finished, move to the next.* So the work expanded to fill whatever time was available, and the programme ended when the budget ended rather than when the corpus was covered. Then somebody asked what had been achieved and nobody could answer, because there was no unit of achievement.

That is what **unbounded** means in practice. Not that the task was hard. That the task had no edges, so nobody could tell how far through it they were.

Six named requirements put edges on it. The six constraints make it **bounded.**

A rule is compiled when all six are answered. It is not compiled when any one of them is not. There is no partial credit and no *nearly*.

And the two states are not the same as what the gate does. The gate has exactly two outputs — allow or deny — and it produces them in milliseconds, thousands of times a day, with nobody watching. Compilation is what happens before any of that: once, upstream, over weeks, with people arguing in a room. An uncompiled rule does not make the gate uncertain. It means there is nothing at the gate at all, and the agent cannot act.

Which means you can finally measure it. How many rules are ready. How many are not. And for every one that isn't, which question is missing and whose desk it sits on.

And you can name the blockage. Not *this policy is too vague* — which is a complaint, and nobody can be assigned a complaint. Instead: *this policy has no authority holder, and Legal has to name one.* Or: *nobody has ever defined what excessive means, and the risk committee has to decide.*

The difference between those two sentences is the difference between a programme that stalls and one that moves. The first describes a condition. The second is a task, with an owner, that someone can finish.

If you want to check this rather than take it, the test is easy and takes five minutes.

Pick a rule from your own organisation. Any rule, but pick a real one. Now try to remove one of the six from it and imagine a machine enforcing what's left. Take away the authority check and watch a perfectly correct decision get made by something with no right to make it. Take away the failure behaviour and imagine the day the authority system is down.

If you can find a seventh requirement that is genuinely load-bearing — not logging, not notification, not escalation, which are all covered above — the list is wrong and this paper wants to know. If you cannot, you have derived it a third time.

One more thing worth noticing, because Chapter 12 depends on it. None of this rests on any particular enforcement technology. The six fall out of the situation — an irreversible act, nobody watching — not out of anyone's product. Whoever builds the gate, wherever they build it, arrives at the same list. That is what makes a compiled artifact portable rather than proprietary.

The next chapter turns the six into a test, and the test into a number you can take to a board.

Chapter 6 — The Assessment

Chapter 5 established what a rule has to become. This chapter turns that into something a team can run on Monday. Before it does, one distinction, because it causes more confusion than anything else in this work.

Compilation is not the same as the decision at the gate. The gate has two outputs, allow or deny, produced in milliseconds, thousands of times a day, with nobody present. Compilation happens long before any of that — once, upstream, over weeks, with people in a room disagreeing. A rule that has not been compiled does not make the gate uncertain. It means there is nothing at the gate to evaluate, and the agent cannot act at all.

The assessment measures the first thing. Executives will hear the second. Say it twice.

The six questions

For any policy you are considering putting in front of a machine, ask these in order.

1. What exactly is being proposed? Name the action. Not the area, the action. *Approve a claim* is an action. *Manage claims* is a department. *Fails when:* the policy governs a topic rather than an event. "All procurement activity shall be conducted with appropriate diligence" has no trigger. There is no moment at which it fires.

2. What does the rule actually say, in terms a machine can test? Reduce it to a condition. A number, a set, a list, a match. *Fails when:* the operative word is a judgment. *Excessive. Material. Reasonable. Appropriate.* Each of those needs a person to say what it means, and until one does, there is no rule — only an intention.

3. Who is allowed to do this? Name the role, and then check that the record naming it actually exists and is current. *Fails when:* the policy says "management may approve" or "an authorised officer." This is the most common failure and the most expensive, because answering it usually means discovering that two departments have held incompatible beliefs about who holds authority, quietly, for years.

4. Can the check finish in time? Every lookup the rule requires has to return, inside the window, every time. *Fails when:* the rule needs an open-ended assessment, an external system with no guaranteed response, or a chain of dependencies nobody has traced. This one is rarely visible on the page. It surfaces when someone tries to implement it.

5. What comes out, and does it bind? The answer has to determine what happens next. *Fails when:* the policy ends in advice. "Escalate as necessary."

"Consider whether to proceed." If the system can carry on regardless of the answer, no rule was enforced.

6. What happens when this cannot be evaluated? The authority record will be down. The rule may not be compiled yet. The lookup will time out. *Fails when:* nothing is written, which is almost always. And note who has to answer it — this is not an engineering decision. Whether the business stops or proceeds when governance is unavailable is a decision at the level of the person who owns the consequence.

A screen that takes ten minutes

Before running the full assessment, there is a faster test that finds most of the same problems.

Print the policy. Highlight every word that came into English from Norman French or Latin. *Appropriate, material, reasonable, excessive, necessary, adequate, substantial, exercise, determine, ensure.*

Those highlights are your ambiguity, nearly every time.[27]

The words that compile are the old ones — the Anglo-Saxon core that describes physical acts with visible actors. *Hold. Stop. Send. Pay. Above. Who signs.* The words that fail are the ones that arrived with lawyers and administrators after 1066, built to describe delegated authority at a distance, and deliberately broad enough to cover cases nobody could list in advance.

That breadth was the point. It is what makes a policy survive contact with situations its author never imagined, and it is exactly what a machine cannot use.

The screen is a rule of thumb rather than a law — you can write vague Anglo-Saxon and precise Latin.[28] But it has a high hit rate, an analyst can apply it on their first day, and it puts the interpretive problem in front of the reader in a form they immediately recognise.

What the assessment produces

Not a pass or a fail. A location. Every policy that does not compile fails at a named question, and each named question routes to a different person.

[27] English carries two vocabularies: a Germanic core describing physical acts with visible actors, and a Norman French and Latin layer that entered through law, church and administration after 1066. The later layer arrived to describe delegated authority exercised at a distance, and its terms retained deliberately broad senses because they had to cover cases that could not be enumerated in advance. That breadth is what makes such terms durable in written policy, and unusable at an execution boundary. The deliberate breadth of Latinate legal terminology, and its function in preserving interpretive room, is treated at length in Mellinkoff, D. (1963). *The Language of the Law*. Little, Brown. On the layered history of the vocabulary itself, see Crystal, D. (2004). *The Stories of English*. Penguin.

[28] The heuristic is offered as a triage device, not a linguistic claim. Precise Latinate terms exist — *exceeds 250,000 United States dollars* — and vague Germanic ones do also — *do right by the customer*. The screen is useful because the correlation is strong enough to direct attention, not because it is exceptionless.

Fails at **Who has	to answer**
What is being proposed The busin	ess owner — is this a rule or a value?
What the rule says Whoever o	wns the standard — usually risk or a domain committee
Who is allowed Legal, an	d whoever maintains the authority record
Can it finish Engineeri	ng, with the domain owner
What comes out The busin	ess owner
What happens when it breaks Whoever c	arries the consequence — often the board

This is the output that makes the exercise fundable. Not *forty percent of your policies are too vague*, which is an insult with a percentage attached. Instead: *two hundred and eleven policies are blocked on question three, and every one of them is waiting on the same thing — a current record of who is allowed to approve what.*

That is a project with an owner, a scope, and an end.

What it also finds

Some policies fail every question, and that is a result rather than a problem.

"Employees must act in the best interests of the company at all times" fails at one, two, four, five and six. There is no triggering action, no testable condition, no way to terminate, no binding output, and no failure behaviour.

It is not a broken rule. It is not a rule. It is a statement of values, it governs people, it belongs in the handbook, and it should never have been sent for compilation in the first place.

Discovering that a policy is not a rule is a finding. A meaningful part of any corpus turns out to be culture wearing the costume of governance, and nobody notices until something tries to execute it.[29]

The next chapter follows what happens to a corpus when this assessment is run across all of it, and where the losses go.

[29]This is an observation from practice rather than a measured finding, and no quantity is claimed. The distinction it rests on is not novel: aspirational statements of value and operative rules have long been recognised as different instruments, and only the second was ever intended to determine a specific outcome.

Chapter 7 — What Happens to Ten Thousand Policies

Run the six questions across a corpus and you get a funnel.

Ten thousand policy statements go in the top. Each question knocks some out. What comes out the bottom — considerably smaller — is the part a machine can actually enforce. The figures that follow are illustrative. No organisation has yet run this assessment across a full corpus and published what fell out, so the numbers show the shape of the attrition rather than measured values.

The Funnel: Figure 1

10,000 policy statements go in…

↓ Q1 is it even a rule?

8,000

↓ Q2 can the rule be tested?

6,200

↓ Q3 who is allowed?

4,700

↓ Q4 does the check finish?

3,300

↓ Q5 does the answer bind?

2,800

↓ Q6 what happens when it breaks?

2,100 executable RULES come out…

Figure 1 — Illustrative attrition through the six questions. Values show shape, not measured data.

The first reaction to that picture is that something has gone badly wrong. Seventy-nine percent lost. A programme that started with ten thousand rules and ended with two thousand looks like a failure by any normal measure.

It isn't. Everything that fell out is the actual work, and this chapter follows where each piece of it goes.

A note on the numbers. No organisation has yet run this assessment across a full corpus and published what fell out, so the figures above are illustrative — they show the shape, not measured values. What is solid is the direction: three things push attrition up, and none has been quantified.[30]

[30]The three: the corpus is larger than the policy count suggests, for the reasons in Chapter 4; judgment terms are pervasive in written policy, because breadth is how policy survives situations its author could not foresee; and authority records are stale in most organisations, because nothing forces them current until something tries to read them. This is the principal empirical gap in the present work, and it is a tractable one. A single institution running the assessment across a defined corpus and publishing the attrition by question would establish the base rate the category currently lacks.

Why it narrows in that order

The questions cannot be asked in any order. Each one only applies to policies that survived the one before, which is what gives the funnel its shape.

Q1 removes the statements that were never rules. Values, principles, mission language, anything with no triggering event. You cannot ask what a rule says until you know when it fires.

Q2 removes the judgment terms. Everything resting on *appropriate*, *material*, *reasonable*, *excessive*. These are not badly written policies. They are policies written for a reader who can interpret, doing precisely what they were designed to do. They fail here because the reader is gone.

Q3 removes the orphans. No identifiable authority holder, or an authority record that turns out to be wrong. This is the slowest stage and the most political, because the answer usually does not exist and has to be created by people who would rather not create it.

Q4 removes the unbounded. A lookup with no guaranteed return, an open-ended assessment, a dependency chain nobody has traced. These look fine on the page. They fail when someone tries to build them.

Q5 removes the advisory. *Consider. Review. Escalate as necessary.* Rules that were never meant to bind and cannot be made to.

Q6 removes almost nothing — but almost nothing arrives with an answer. The box is empty on nearly every survivor, and filling it in requires a decision most organisations have never had to make.

The funnel narrows fastest at the top and slows as it descends. The cheap losses come first. The expensive ones come last.

Where the losses go

Nothing that falls out disappears. Every stage produces work, and each kind goes somewhere different.

Fell out at		
**Goes to	** **What they have t	o do**
Q1 — not a rule Business	owner Confirm it belongs	in the handbook, governing people
Q2 — untestable Risk, or	the domain committee Decide what *exces	sive* means. In a number. In writing.
Q3 — no authority Legal, pl	us whoever owns the authority record Settle who signs —	usually for the first time

Fell out at		
**Goes to	** **What they have t	o do**
Q4 — cannot finish Engineeri	ng, with the domain owner Guarantee the depe	ndency, rewrite the rule, or withdraw the action
Q5 — does not bind Business	owner Answer one questio	n: did you mean this to bind?
Q6 — survivors Whoever c	arries the consequence, often the board Decide what happen	s when governance is unavailable

Look at that column of owners. Not one of them is a technology team.

Why the losses are the product

The funnel found every place where the organisation's rules were ambiguous, and named the ambiguity. It found every place where nobody knew who was in charge, and forced an answer. It found the rules that had quietly stopped applying years ago. It found the exception pathways that were governing real decisions while existing in no document. And it found the situations nobody had ever planned for, because until now nothing made them plan.

None of that was known before. All of it was true before.

The exercise did not create those problems. It made them visible, in a list, with owners attached. The organisation was already running on ambiguous authority and undocumented practice. It was simply doing so without a record.

Which reframes what this is. Not a technology programme that happens to surface governance problems — a governance modernisation programme that produces executable rules as a by-product. And the by-product is the smaller half of the value.

It is also why this cannot be bought as software. A tool can run the six questions. A tool cannot make Legal and Finance agree about who signs.

The number that goes to the board

One figure comes out of all this.

Of the policies governing what our agents are permitted to do, this many are executable. The rest are not. Here is what is blocking each one, and whose desk it is sitting on.

That is a coverage figure. It is measurable, it moves quarter over quarter, and it is the only honest answer to the question the insurer and the regulator are already asking.[31]

[31] See Chapter 2. The demands described there — from insurers pricing an exposure they cannot model, regulators moving from whether a policy exists to whether it applied, and counterparties seeking a verifiable artifact — all reduce to the same question about what was enforced at the moment of action. A coverage figure derived from the six questions is the only form of answer that can be produced in advance rather than reconstructed afterwards.

The next chapter follows a single policy all the way through — from the sentence in the binder to the four milliseconds at the gate — and shows what the four weeks in between actually consisted of.

Chapter 8 — Four Weeks and Four Milliseconds

The funnel is an aggregate. It tells you what happens to ten thousand policies and nothing about what happens to one.

This chapter follows a single policy the whole way through. It is worth reading slowly, because everything this paper claims about cost, ownership, and where the human work goes is visible in one example.

The claim that did not fit

A health insurer processes claims against a benefit table.[32] The table holds every treatment code the insurer covers and the maximum payable against each. If the code is in the table, the claim can proceed. If it is not, the system blocks.

Medicine does not wait for the benefit table. New procedures appear. Diagnosis codes get revised. Treatments arrive that map to nothing in the table because the table was last updated eighteen months ago and the committee meets quarterly.

So an assessor meets a claim with a code that is not there. She does not stop the claim. She finds the nearest thing in the table that covers roughly the same treatment, and processes under that.

The system produces a clean output. Table-validated. Approvable. Nothing flagged.

The record shows a claim that cleared validation. What actually happened was a person deciding that one treatment was close enough to another, on her own judgment, using knowledge she had built over nine years. The original code is not in the record. Neither is the mapping. Neither is her reasoning.

Nobody did anything wrong. This is how the department has worked for a decade, it is why claims get paid on time, and every assessor on the floor does it. It has simply never been written down, because nothing ever required it to be.

That is the exception layer from Chapter 4, caught in the act.

[32]The claims environment described here is a constructed composite drawn from practice in regulated insurance operations. It is illustrative of a pattern rather than a report on any particular organisation. The pattern — human interpretation resolving gaps between a reference table and operational reality, leaving no trace in the record — is common wherever a controlled vocabulary is maintained on a slower cycle than the practice it describes.

The same claim, now

The insurer deploys AI agents on routine claims. Forty thousand a day, straight through, no human reading them. The policy the agents work from is one sentence. *Approve routine claims up to the standard limit.*

Nine years an assessor read that sentence and knew what to do. The agent reaches a claim with an unrecognised treatment code and stops, because it has three questions the sentence cannot answer.

What counts as routine? Every claim that cleared the table was routine, historically — including all the ones that only cleared because somebody quietly mapped them. The word *routine* has been doing work nobody knew about.

Who is allowed to decide a mapping? Nobody, formally. That authority does not appear in any document. It was never granted. It accumulated. When the question was finally asked out loud, it turned out that medical coding believed they owned mappings, and claims operations believed assessors did, and both had been right in practice for years.

And what happens when the benefit system is unreachable? Nobody had asked. When it went down, assessors worked from memory and reconciled later.

At forty thousand a day, none of those has a human answer available. There is no version of this where somebody phones the coding team forty thousand times.

The four weeks

Two workshops with medical coding, claims operations, and legal. One session that ran three hours over.

The mapping question was settled first, and settling it changed the operating model rather than the software. Mappings would no longer be made at the moment of the claim. They would be defined in advance by medical coding, with an explicit list of which codes may substitute for which. An unrecognised code with no defined mapping does not get approximated. It routes to a human.

Routine was defined by exclusion, which is what usually happens. Not *here is what routine means* but *here is what is definitely not routine* — anything above the standard limit, anything with an undefined code, anything on the fraud watch list, anything from a provider under review. Everything else is routine by default.

The third question went to the COO, because nobody below her had the standing to answer it. If the benefit system is unreachable, do forty thousand claims a day stop, or do they proceed unchecked?

She chose stop. Then she asked how long the benefit system had been going down for, and discovered it was three times that year, and started a separate programme about that.

Notice what those four weeks were not. No code was written. No model was selected. The time went on getting people with authority into a room and keeping them there until they decided things nobody had ever had to decide.

What came out

policy_id: CLM-017

action: claim_approval

standard:

amount: { at_or_below: standard_limit }

treatment_code: { in: benefit_table }

mapping: { must_be: predefined }

routing:

undefined_code: HUMAN_DEFAULT

above_limit: HUMAN_DEFAULT

provider_under_review: HUMAN_DEFAULT

authority:

registry: benefits.authority.v2

mapping_owner: medical_coding

binding_output:

admissible: ALLOW

inadmissible: DENY_ROUTE_TO_ASSESSOR

failure_behaviour:

benefit_system_unavailable: HALT_AND_ESCALATE

evidence: required

Twenty-two lines.

Four weeks to write them, and the four weeks were not spent on the twenty-two lines. They were spent discovering that a large insurer did not know who was allowed to decide what a treatment was equivalent to — and had been running on that gap, successfully, for over a decade.

Forty thousand times a day

09:14:22.118 AGENT propose claim_approval CLM-88392

09:14:22.118 GATE standard matched CLM-017

09:14:22.118 GATE amount within limit

09:14:22.119 GATE treatment code in table

09:14:22.119 GATE ALLOW

09:14:22.119 WITNESS recorded

09:14:22.120 AGENT execute — payment released

09:14:22.204 AGENT propose claim_approval CLM-88393

09:14:22.204 GATE standard matched CLM-017

09:14:22.204 GATE treatment code not in table

09:14:22.205 GATE no predefined mapping

09:14:22.205 GATE DENY — route to assessor

09:14:22.205 WITNESS recorded

09:14:22.206 AGENT halt, queue for human review

Two claims, eighty-six milliseconds apart. One paid. One sent to a person.

The second one is the important one. Under the old system that claim would have been approximated into the table and paid, and no record would exist of the judgment that made it payable. Now it stops, and a human makes the same decision she always made — except this time the decision is the record rather than something that happened before the record started.

The governance took under two milliseconds. It will take under two milliseconds forty thousand times today, and again tomorrow.

What this example is actually showing

Three things, and each one generalises.

The cost is entirely upstream. Four weeks of expensive human argument, then a permanent asset that costs almost nothing to run. Every economic property of this category follows from that shape. It is why the services half is large and the software half is thin. It is why the first policy is expensive and the hundredth is cheap. It is why nobody can sell you this as a licence.

The agent was never the problem. It behaved correctly throughout. It approved what was clearly approvable and stopped at what was not. What it could not do was invent an answer to a question the organisation had never settled — and that is the entire failure mode. It has nothing to do with model capability, and a better model would not have helped.

One sentence failed three of the six questions. Six words, in use for nine years, that nobody thought was ambiguous. It was ambiguous the whole time. The ambiguity was being absorbed, silently, thousands of times a week, by people who had stopped noticing they were doing it.

Multiply that across a corpus and Chapter 7's funnel stops looking pessimistic.

The other outcome

Not every policy survives, and the failure looks different from what you would expect.

Employees must act in the best interests of the company at all times.

That fails five of the six questions. No triggering action, no testable condition, no terminating evaluation, no binding output, nothing to fall back to. Establishing it took two days, and almost none of that was analysis. It was spent finding someone willing to say out loud that the sentence should never have been sent for compilation.

That is the harder half. Identifying a values statement is trivial. Getting an institution to agree that one of its governing sentences governs nothing a machine can reach requires someone with the standing to say so, and standing is what the exercise is short of throughout.

The next chapter takes what the funnel left behind — everything that did not compile — and asks what happens to an organisation that has to run on both halves at once, permanently.

Chapter 9 — Both Halves, Permanently

Act One — the claims floor you have now

US health insurers process around three billion medical claims a year.[33] One large carrier alone handled 6.4 million claims in a single product line — the ACA marketplace — in one year.[34]

Nobody reads those. They cannot. A claims floor of any size clears its volume by letting the great majority pass automatically: code is in the table, amount is under the limit, provider is known, straight through, paid.

What the assessors handle is the residue. An unrecognised treatment code. An amount needing an override. A provider flagged last month. Those get read properly, sometimes twice, sometimes with a walk over to someone who has been there longer. The floor has a rhythm — volume peaks after the weekend, a queue by Thursday, team leaders moving people around to clear it. Everybody knows who to ask about oncology codes. Everybody knows Liz is the one who actually understands the old policies. None of that is in a process document. So here is the first thing to notice, and it undercuts the most common objection to everything in this paper.

Machine-speed adjudication is not coming. It has been here for years. Most claims are already decided *without* a person reading them. The gap this paper describes is not a future problem created by agents. It exists right now, at the volume above, in systems that have been running since before anyone said the word agentic.

[33] 2024 CAQH Index Report, February 2025 , based on three billion medical claims filed by 216 million health plan members.

[34] CMS Transparency in Coverage Public Use File, Plan Year 2024, analysed by MoneyGeek (January 2026). UnitedHealthcare processed approximately 6.4 million ACA marketplace claims in plan year 2024. Figures cover in-network, post-service claims only.

And it is not benign. Denial rates are climbing — 41% of providers report their claims denied more than one time in ten.[35] The largest carrier denies around one in five in-network marketplace claims, roughly 1.28 million in a year, and its Medicare Advantage prior-authorisation denials sit at 12.8%, the highest among major carriers.[36]

Now ask the question that matters. Pick one of those denials. Any one.

Which rule was applied? Who was authorised to set that rule? What did the system do when the code was not in the table — and who decided that was acceptable? The record shows an outcome. It does not show the decision that produced it, because the decision happened before the system started recording.

And notice that adding people does not fix it. The assessor was there the whole time. She mapped the code, the claim cleared, and nothing captured that a judgment had occurred. Hire a hundred more and you scale the invisibility. The problem was never that there was no human in the loop. It was that the loop was invisible.

That is the state of things today, at human speed, with the current book of business.

Act Two — the floor after compilation

Walk in on a Tuesday and the first thing you notice is that it's quiet.

The queue board on the wall is gone. There's nothing to display. Claims arrive and clear, arrive and clear, and the only sign of it is a number ticking over in the corner of somebody's screen. No Thursday backlog. No team leaders moving people between desks. No peak after the weekend, because the thing doing the work doesn't know it's a weekend.

Most of the floor is empty. Two hundred and twenty desks became thirty, and the thirty cluster at one end because it felt strange to spread out.[37]

Liz is still here. She's just not assessing claims any more.

She's upstairs two days a week with someone from medical coding and a man from a governance team nobody had heard of eighteen months ago. New device, new technique, a code the committee hasn't got to. Liz's job is to say what it may be treated as. The man from governance asks her the questions that turn her answer into something the gate can enforce — what exactly triggers it, who's allowed to change it, what happens if the mapping table can't be reached. He doesn't know about oncology. She doesn't know why he keeps asking about the third one.

[35] Experian Health, *2025 State of Claims Report*. Forty-one percent of providers report claims denied more than ten percent of the time, with denial rates rising, attributed in part to increasingly complex payer requirements and more aggressive claims review. Note that denial rate measures friction between payer and provider, not adjudication accuracy; a denial may be correct.

[36] CMS Transparency in Coverage Public Use File, Plan Year 2024, analysed by MoneyGeek (January 2026). UnitedHealthcare's ACA marketplace denial rate was approximately 20 percent — roughly 1.28 million denials — and its Medicare Advantage prior-authorisation denial rate 12.8 percent, above Aetna (11.9 percent) and Humana (5.8 percent).

[37] Headcount, per-assessor throughput, and the split between gated and human-routed volume are constructed to show the shape of the change. Claims-per-assessor figures are not published, and audited adjudication error rates are not available. The industry and single-carrier volumes cited earlier in this chapter are real; the operational detail illustrating them is not.

She used to do this forty times a week without noticing. A claim would arrive with a code that wasn't in the table, she'd find the nearest thing that was, and the claim would go through. Ninety seconds. No record that a judgment had been made at all.

Now she does it once, before any claim arrives. One code, one afternoon, one argument about whether the substitution actually holds — and it comes out as a document with her name on it. Every claim carrying that code from then on is handled by her decision, and she never sees any of them.

Why Liz, and not someone cheaper.

Every automation programme before this one displaced the experienced first. This one cannot, and the reason is structural: the input it needs is the knowledge that was never written down, and only the people holding it can supply it. Liz is in that room because she is the only person who knows why the mapping rule reads the way it does and which parts of it were never actually enforced. Twenty-two years of knowing where the bodies are buried is not a soft asset here. It is the input.

Two other people are in the room with her. One from governance, whose job is to turn her answer into something the gate can enforce. One with the standing to decide when the first two disagree. Chapter 11 takes all three properly.

Downstairs, the thirty.

They only see what the gate stopped. Undefined codes with no mapping. Amounts over the limit. Providers under review. The claims where something didn't fit.

Ask one of them how the job changed and she'll say there's nothing easy left. She used to work a stack where most cleared in under a minute and the hard ones came as a break in the rhythm. Now every one is a hard one. She gets through fewer and finishes more tired.

She'll also mention, if you ask, that she doesn't see normal claims any more — so when something odd turns up she has less to compare it against than she used to.

And one thing on the floor that didn't exist before.

A dashboard near the front showing how many claims went to people yesterday, and the day before. Somebody's job is to watch it. When the number moves, they find out why — a new provider whose codes don't map, a product launched last month, a rule that has quietly stopped matching what the business actually does.

Nobody had that job before, because nobody could see the number. The queue was just the queue.

What actually changed.

Before, most claims cleared without a person and the record showed that they cleared. Now most claims clear without a person and the record shows which rule matched, which conditions were satisfied, what authority the agent held, and what the gate decided.

Same volume of unattended decisions. Entirely different question answerable afterwards.

And Liz's decision about the treatment code — the one that used to take ninety seconds and vanish — is now written down, dated, attributed, and superseding a previous version.

That is the whole change, and it is larger than it sounds.

Act Three — the floor nobody has seen yet

Here is what the executive reading Act Two expects next: compile more, the exception queue shrinks, the thirty become five. It does not go that way, and the reason is economics rather than technology.

Every insurance product has a floor set by what it costs to adjudicate a claim against it. If assessment costs eight dollars, no product with an expected claim value of six dollars can exist. That floor has quietly shaped the entire industry — it is why cover is sold annually rather than per event, why policies are broad rather than specific, why small claims get bundled, excluded, or simply not offered.

Drop adjudication cost toward zero and the floor drops with it.

This is Jevons' paradox, and it is one of the most reliable patterns in industrial history.[38] Making coal-fired engines more efficient did not reduce coal consumption. It increased it, because cheaper steam made economical a whole range of things that had not been worth doing. Efficiency in a valuable input does not shrink demand for that input. It expands the set of viable applications until total consumption rises.

The input here is adjudication. Make it nearly free and products that were never viable become viable — each one carrying far lower claim value and therefore far higher claim frequency per dollar of premium.

Nobody can tell you what those products will be. That is not a gap in this argument. It is the argument. If we knew, we could write the rules now. We do not, so we cannot. What we know with confidence is that there will be new insurance products, because the constraint that prevented them is being removed, and removed cost constraints have never once failed to produce new opportunities.

Nor is any of it optional for a single carrier. The moment a competitor sells per-procedure cover profitably, everyone must, and the volume arrives whether or not your compilation function is ready.

Then the arithmetic closes the human option permanently. Not because machines are faster — because there is no headcount that reaches the volume, and the claim values no longer support the labour cost even if there were. The economics that make the new products sellable are the same economics that forbid a person in the loop.

[38] Jevons, W.S. (1865). *The Coal Question: An Inquiry Concerning the Progress of the Nation, and the Probable Exhaustion of Our Coal-Mines*. Macmillan. Jevons observed that improvements in the efficiency of coal use increased rather than reduced total consumption, because cheaper steam power made economical a range of applications that had not previously been viable.

And the demand for compiled governance rises on exactly the same curve. Every novel product needs its own rules, limits, exclusions, and failure behaviour. More products means more policy. More policy means more questions that only people with authority can answer.

Which is Chapter 1's asymmetry, accelerated by its own success. The better compilation gets, the more governance the business generates, and the further behind the interpretive bottleneck falls. The compilation function is not a project that finishes. It is standing capacity, sized for a corpus that has not been written yet, covering products nobody has invented.

Why both halves are permanent

Two reasons, and they compound.

New rules arrive faster than old ones compile. The regulator publishes. A product launches. Medicine invents a procedure. Every one of those is the business doing something — entering a market, shipping a product, covering a treatment it couldn't cover before.

Act Three means more of it, not less. That is what growth looks like from the compilation function's side of the desk.

Which is why this is capacity rather than a project. You are not funding a translation of the policies you have. You are funding the ability to translate whatever the business writes next — quickly, repeatedly, for products nobody has designed yet. An organisation that can compile a new product's rules in three weeks can launch things an organisation that takes nine months cannot.

And some of the exception queue was routed to humans on purpose**.** An undefined code goes to an assessor not because nobody has got round to compiling it, but because deciding one treatment is equivalent to another is a clinical judgment the organisation chose not to hand to a machine. That routing does not expire. Ten years from now it will still be there, and it should be.

	Decided by the gate	
	**Decided	by a person**
Speed Milliseco	nds Minutes to days	
Volume Very high	, and rising Low proportionally	, never zero, rising absolutely
Authority	eeks ago, in a room	oment
Settled w	Exercised in the m	
Cost per decision Near zero	Unchanged	
What arrives	g the rule covers	ly does not, plus what nobody has
Everythin	What it deliberate	answered yet

The line between them moves. Compile more and it shifts one way. Launch a product and it shifts back. It never reaches either end.

The dangerous half

Once most volume flows through the gate without incident, attention follows the volume. The dashboard shows throughput. The board hears about coverage. Investment goes toward compiling more.

Meanwhile every hard case in the business now lives in the exception queue. Every ambiguous claim, every unrecognised procedure, every provider under review — concentrated into one function, worked by a team whose headcount was justified by falling volume.

The organisation has gathered its hardest decisions into its smallest team and stopped watching them.

The gate will be fine. The gate is deterministic and does what it was told. The risk moved to where people are still deciding — fewer of them, each decision heavier, and nobody reporting on it because the number looks small.

It is also where the litigation lives. The claims that get contested are not the routine ones the gate approved. They are the ambiguous ones, the denied ones, the ones where somebody exercised judgment. Those are now all in the same queue, handled by the smallest team, and if that team's reasoning is no better recorded than the assessor's was in Act One, nothing has been fixed. It has been concentrated.

The next chapter takes what happens when the check cannot run at all, and shows why the answer differs by industry while the structure underneath does not.

Chapter 10 — The Morning the Table Went Down

09:14 AM on a Tuesday. The benefit table stops responding.

Nothing dramatic happens. No alarm on the claims floor, no error on anybody's screen. Somewhere in a data centre a service that has been up for eleven weeks quietly isn't, and the first person to know is a monitoring system talking to another monitoring system.

Claims keep arriving. They arrive at the same rate they always do, because the outside world has no idea.

The gate takes the first one at 09:14:03. It matches the standard, checks the amount, and goes to look up the treatment code. The lookup does not return. Not *the code isn't there* — that has an answer. The table itself cannot be reached, so there is no answer to have. The gate has a proposed action, no way to determine whether it is permitted, and nobody to ask.

What happens next

It stops. Then it does it again, four hundred and twelve times in the first eleven minutes.

Downstairs, the queue starts to fill. The dashboard near the front — the one somebody's job is to watch — reads four hundred at 09:20. Eleven thousand by 09:34. The thirty assessors are looking at a number that has never behaved like that.

Someone calls the COO at 09:41.

And nothing is broken.

The gate is doing precisely what the COO decided it would do, in a room, four weeks earlier, when somebody from governance asked what should happen if the benefit system were unreachable and she realised nobody had ever asked. She said stop. Hold everything and escalate.

So it stopped. Eleven thousand claims are held rather than paid, the floor is in uproar, and the system is behaving correctly. *The chaos is the policy working.*

At 09:47 the table comes back. The held claims flow through in under a minute. The record shows: thirty-three minutes, eleven thousand four hundred and six proposed actions, none executed, each one held with the reason attached.

The same morning, two years earlier 09:14. The benefit table stops responding.

The assessors notice within a minute or two, because the screen hangs. They wait. It doesn't come back. Somebody says the table's down again. And then they carry on.

They know most of the common codes. They know roughly what the limits are. Claims have to go out today, the phones are still ringing, and the reconciliation can happen later — it always does. So for thirty-three minutes, several hundred claims are assessed against a benefit table that nobody could actually read, by people working from memory, doing their best.

Most of them are right. Some of them are not.

At 09:47 the table comes back and the floor carries on. Nothing is flagged. The daily report shows a normal Tuesday. IT logs an outage against a service, and nobody connects it to anything.

Ask, six months later, which claims were paid during those thirty-three minutes and whether any of them should have been. Nobody can tell you. Not because someone is hiding it — because the question was never capable of being answered. The record does not distinguish those claims from any others, and the people who processed them have handled forty thousand more since.

The difference One morning was loud and left a complete record of exactly what was ungoverned, for how long, and what was done about it. The other was quiet and left nothing.

The quiet one is the dangerous one, and that is the whole point of the sixth question. Not that systems fail — systems fail. It is that failure without a declared behaviour

produces a hole in the record that nobody can characterise afterwards, and the holes tend to occur precisely when things were least under control.

Stopping is not the only right answer

The COO chose to stop. In her business that was correct: money that moves cannot be recalled, and holding a claim for thirty minutes harms nobody.

Change the business and the answer changes with it.

A **clinical decision support** system that loses its constraint service stops advising, and the clinician decides unaided — which is what she did before the system existed, so the fallback is a known state rather than a novel one.

An **industrial interlock** that cannot establish admissibility blocks the transition and brings the equipment to a safe state. Stopping the line costs money. Not stopping it costs something else.

A **grid, or a hospital system, or a broadcast chain** does the opposite. Governance unavailable, and it continues — under a pre-authorised degraded mode with a restricted set of permitted actions, because the harm from stopping exceeds the harm from operating ungoverned for a bounded period. Two stop. One hands back to a person. One carries on.

Those directions contradict each other, and all four are right. Anyone claiming that governed systems must universally fail one way has not looked at more than one industry.[39]

What is actually universal

Not the direction. Five properties underneath it, present in all four.

The behaviour was **decided in advance** — chosen when there was time to think, not discovered at 09:14.

Somebody **with standing** chose it. Not the developer who wrote the timeout handler three years ago and thought failing the request seemed harsh. In Chapter 8 it went to the COO, and that was the right altitude.

The machine **does not invent authority**. Whatever it does, it does because it was told to. It does not conclude that proceeding is probably fine.

Authority **resolves somewhere specific** — to a person, to a safe state, to a restricted mode with a defined scope. Somewhere named, not nowhere.

And the transition is **recorded**: when governance became unavailable, what happened instead, and for how long.

[39] Fail-state conventions differ by sector and are established in each sector's own standards literature. The safe-state concept in industrial control is defined in IEC 61508, cited at Chapter 1. Nothing in the argument of Chapter 5 bears on failure direction; the six requirements fix that a failure behaviour must exist and must be chosen in advance, not which behaviour is correct for a given domain.

Call it **Declared Failure Sovereignty**. Sovereignty, because the question is where authority sits when the mechanism holding it stops. Declared, because the answer exists before it is needed — or it does not exist at all.

The default nobody chose Here is what the sixth question is really protecting against.

The gate cannot evaluate. Something happens next; something always does. Maybe it proceeds, because of a timeout handler written by a contractor in 2023. Maybe it blocks, and a production line stops at two in the morning for reasons nobody can explain until Tuesday.

Either way, the organisation's behaviour under its worst conditions was set by accident, at a level with no standing to set it, and probably nobody in the building knows what it is.

A policy that has answered five of the six questions is not nearly compiled. It is a rule with no behaviour under the conditions where behaviour matters most — and it will get one anyway, from whoever wrote the timeout handler.

One more thing about that Tuesday When the COO was asked what should happen if the benefit system were unreachable, her first response was to ask how often that actually happened.

Nobody knew. Somebody went and found out: three times that year, for a total of about four hours.

Four hours of claims processed against a table nobody could read, and no record of which ones. That had been true every year for a decade and had never been visible, because until the question was asked out loud there was no reason for anyone to ask it.

The sixth question found that. Not the gate, not the tooling — the question was asked because somebody had to ask it. Which is worth stepping back from, because a room full of people deciding in advance what a rule means is not how governance has ever worked before.

Interlude — Ten Thousand Years of Deciding Afterwards

Somewhere around 2100 BC, a scribe in Ur pressed a set of rules into wet clay. If a man knocks out another man's tooth, he pays two shekels. If he divorces his wife, he pays a mina — sixty shekels. The Code of Ur-Nammu is the oldest surviving law we have,[40] and it is written in exactly the form law has taken ever since: a list of situations, and what should follow.

Which means it was already incomplete on the day it was written.

[40] Code of Ur-Nammu, c. 2100–2050 BC, Third Dynasty of Ur. The surviving tablets are fragmentary and the reconstruction is partial. The provisions cited correspond to articles 19 and 6 in Roth's translation; the divorce provision reads one mina of silver, which is sixty shekels in Mesopotamian sexagesimal metrology. Attribution of the code to Ur-Nammu rather than to his son Shulgi is contested. See Roth, M.T. (1997), *Law Collections from Mesopotamia and Asia Minor*, 2nd ed., Society of Biblical Literature.

No list of situations covers the situations. A man knocks out a tooth in a fight he did not start. A man divorces a wife who has already left. The clay says what happens; it does not say whether this is that. Somebody had to decide, and that somebody was a person, sitting in a room, after the fact, looking at the parties in front of him.

Four thousand years later we are still doing it.

Every institution we have built resolves ambiguity afterwards, because until very recently there was no other option. The only intelligence capable of judging a hard case was human, and a human cannot be present at every transaction. So we built the review to happen later — and then built entire professions around the later.

A contract is interpreted when disputed, not when signed. Accounts are audited after the year closes. Regulators enforce against conduct that has already occurred. Courts operate entirely in the past tense: every case is about something that already happened, argued by people who were not there when it did.

That is not a flaw in the design. It is the design. Ambiguity is permitted at the moment of action, and resolved at leisure by someone with standing. It has worked for four millennia, and it produced a professional class whose expertise is precisely this: reading what was written, deciding what it meant, and doing so long after the event.

In 1066 the arrangement acquired a second language.

The Normans did not abolish English law. They administered it in French, and pleaded it in French, for three hundred years — and in the written record for considerably longer.[41] — while the population it governed went on speaking English. A vocabulary arrived that existed to describe authority exercised at a distance: *appropriate*, *reasonable*, *material*, *sufficient*, *proper*. Words with deliberately wide edges, because a lord's steward in a county he had never visited needed language that would stretch to cover whatever he found there.

That breadth was an achievement, not a failure. It is how a rule written once could govern cases nobody had imagined. English still carries both layers[42] — the old words for doing things, the newer ones for authorising them — and every policy binder in every organisation is written in the second.

It is also, as Chapter 6 showed, exactly the vocabulary that will not compile.

Here is what has changed, and it is a small change with a very large consequence.

Machine-speed execution does not remove the review. It removes the *interval the review lives in*. There is no later. The action completes, the money moves, the claim pays, and there is no point after the event at which a person can look at it and decide whether it was right — because by then it is simply what happened.

A system that cannot resolve ambiguity afterwards has to eliminate it beforehand.

[41] The Pleading in English Act 1362 (36 Edw. III c. 15) required that oral pleadings in the English courts be conducted in English. It did not end the use of Law French, which persisted in legal education, in the Year Books, and in printed law reports until the Proceedings in Courts of Justice Act 1730 (4 Geo. II c. 26).

[42] On the two vocabularies and their different functions, see Chapter 6, note 27, and the sources cited there.

That is the shift. Four thousand years of governance built on *decide later* meeting a class of execution where later does not exist. Every consequence in this paper falls out of it. Compilation is what eliminating ambiguity beforehand actually looks like in practice. The six questions are the form the elimination takes. The Uncompiled Region is everything we have not yet eliminated it from — and where, for now, the old arrangement still holds.

Be careful about how far to take this. Society is not becoming deterministic. Most of human life will remain interpretive, and should. Chapter 9 spent three acts arguing that The Uncompiled Region never closes.

What is new is that a deterministic region now exists at all — that some part of institutional governance has moved permanently from *decide later* to *decided already*, and that region is growing. It is the first time in the history of written rules that the interpretation has had to happen before the case rather than after it.

Which raises the question the next chapter is about.

The professions built for the old arrangement — the lawyers, the auditors, the compliance officers, the assessors — were not built to write rules. They were built to interpret them, after the event, when the facts were in. That skill has not become worthless. But the moment at which it is exercised has moved, from after the action to before it, and from a case in front of you to a case that has not happened yet.

That is a harder job than the one they trained for. It is also the job the arrangement now requires, whether or not anyone was trained for it.

Chapter 11 — The People Who Move Upstream

Liz has twenty-two years of knowing which treatments can stand in for which.

None of it is written down. She could not write it down if you asked her to — not in any useful form. Ask her how she decides and you get *well, it depends*, followed by four examples, followed by an exception to the third one. The knowledge exists as a very large number of specific cases and a sense of where the edges are.[43]

That is what expertise looks like from the inside, in every field. And it is exactly what the compilation programme needs, because the knowledge is nowhere else.

[43]Polanyi, M. (1966). *The Tacit Dimension*. Garden City, NY: Doubleday. The formulation that we can know more than we can tell is the standard reference for knowledge that resists articulation. This is the same constraint identified as the knowledge acquisition bottleneck in Chapter 3, described from the holder's side rather than the elicitor's.

Why this inverts every previous automation wave

Every prior wave displaced the experienced first. The logic was always the same. Experience is expensive. The work being automated was routine. Routine work went, and the people who had done it longest were the most costly to keep. Cheque processing, telephone exchanges, typing pools, back-office reconciliation — the pattern held for a century.[44]

This one runs the other way, for a structural reason.

The scarce input is not labour. It is knowledge that exists in no document, held by people who have been somewhere long enough to have accumulated it. The interlude explained why: an interpretive system does not need its rules written down completely, because a person fills the gaps at the point of use. Those gap-fillings are the operating knowledge, and they live in individual heads.

A declarative system cannot run on that. Everything has to be stated in advance. Which means the tacit has to be made explicit — and the only people who can supply it are the ones holding it.

The junior analyst can operate the tooling. The junior analyst cannot answer question three.

The four professions, relocated

The interlude named them: lawyers, auditors, compliance officers, and the domain practitioners like Liz. All four were built for retrospective resolution. All four move, and the move is the same in each case — from after the event to before it.

The auditor tested whether a control operated as described. That test happens after the period, on a sample, looking for exceptions. Upstream, the same person specifies what the control must be — what condition, what threshold, what evidence — before anything runs against it. The skill transfers almost directly: an auditor's core competence is knowing what would constitute adequate evidence that something happened, and that is question five in a different accent.

The lawyer argued what a term meant when the parties disagreed. Upstream, she decides what it means, in advance, in writing, before anyone relies on it. This is a harder job and worth being honest about why: interpretation after the fact is bounded by a specific dispute with known facts. Definition in advance has to survive every case that might arise, including the ones nobody has thought of. She is drafting without a record.

The compliance officer investigated breaches. Upstream, he defines what would constitute one — which sounds like the same thing and is not. Investigating a breach means

[44] On the general displacement pattern, see Autor, D.H. (2015). Why Are There Still So Many Jobs? The History and Future of Workplace Automation. *Journal of Economic Perspectives*, 29(3), 3–30 . The claim in the text is not that experienced workers were always displaced first, but that automation has historically substituted for codified, routine tasks while complementing tacit and judgment-intensive ones. The inversion described here follows from the fact that compilation requires the tacit component as its input rather than displacing it.

reasoning backwards from harm. Defining one means reasoning forwards from a rule, to a condition a machine can test, before any harm exists to reason from.

And the domain practitioner — Liz — supplies the substance the other three shape. She is the only one who knows what the answer actually is.

The three roles that emerge

Chapter 9 introduced them from the floor. Here they are properly.

The Domain Authority. Holds the knowledge. Has standing to say what the answer is. Does not need to understand compilation and should not be asked to — the moment you require a domain expert to learn the six questions, you have narrowed your pool to people who possess both, and there are almost none.

The Constraint Architect. Knows what the six questions require and how to extract an answer that will hold. Part analyst, part interviewer, part draftsman. The nearest existing title is senior business analyst, which undersells it — the distinguishing skill is knowing that *we handle that case by exception* is not an answer, and asking again, and continuing to ask when the room wants to move on.

This role is new, or nearly. Business analysis comes closest, requirements engineering is adjacent, and legal drafting shares the discipline of writing something that has to survive contact with cases you cannot foresee. But none of them is quite it, and there is no qualification for it yet. There will be.

The authority holder. Decides when the room disagrees. Legal for who signs. Risk for what a threshold means. The COO for what happens when the system is down. Not in the room full time — in it for the calls nobody below them can make. This is the role that most often stalls a programme, because their diary is the constraint and nobody can proceed without them.

Notice who is absent. Nobody in that list is writing code. The engineering is downstream and comparatively trivial, which is the single most counter-intuitive staffing fact about this work.

The honest part about headcount

This paper is not going to tell you that automation preserves jobs. It does not, and pretending otherwise would discredit everything else here.

Two hundred and twenty assessors became thirty, plus a handful upstream. That is a large reduction and it is permanent.

What changed for the ones who remain is the weight of each decision. Liz's judgment used to be consumed one claim at a time, ninety seconds each, invisible, and if she got one wrong it affected one claim. Now it is written down once and applied to every claim that follows. If she gets it wrong, it is wrong at volume, for months, until somebody notices.

She also decides years before the consequence lands, about cases that have not happened, with no ability to correct at execution because there is nobody there. Her name is on the document.

Fewer people. Much more consequence each. That is the trade, and it should be stated plainly rather than dressed up as empowerment.

The window

Here is the part that makes this urgent rather than merely interesting.

The knowledge leaves when the person does.

An organisation that starts this work while its experienced cohort is still employed can extract what they know. An organisation that starts ten years from now, after they have retired, will find its rules cannot be reconstructed — because the people who understood why they existed are gone, and the documents were never sufficient on their own. They never had to be. That was the whole arrangement.

This is not a technology timeline. It is a demographic one, and it is the strongest argument for doing this now rather than when the tooling improves. The tooling will improve. Liz will not still be there.[45]

What organisations will actually do

They will hire for the wrong thing first. Almost everyone does.

The instinct is to staff this with technologists, because it looks like a systems programme and produces a machine-readable artifact. That staffing produces beautifully engineered rules that encode the wrong answers, because nobody in the room had the standing to say what the answer was.

The second instinct is to staff it with consultants who know the methodology but not the business. Better — they will run the six questions properly — but they cannot supply what Liz supplies, and the engagement stalls at question two until somebody internal is pulled in.

The pattern that works is uncomfortable to fund: a small number of expensive, long-tenured internal people, taken off their day jobs, sitting with a Constraint Architect for as long as it takes. Their absence is felt immediately in operations. The value of their presence in the room shows up eighteen months later.

That is a hard business case, and it is the actual one.

The next chapter turns from the people to the artifact they produce, and asks a question with a great deal of money riding on it: once a rule has been compiled, can it be moved?

[45] The demographic claim is asserted rather than sourced. Retirement projections for the relevant professions — insurance claims, audit, compliance — would support it if available; absent that, the claim should be read as an argument about tacit knowledge transfer rather than a workforce forecast.

Chapter 12 — What You Actually Bought

Four weeks for one rule. Two hundred rules, and you have spent a year of expensive people's time, most of it in rooms, arguing.

The natural way to book that is as project cost. Something was implemented, the implementation is finished, the money is gone.

That is the wrong way to think about it, and the difference matters, because an organisation that treats compilation as project cost will underfund it, deprioritise it, and be unable to explain to anyone what it got.

It is not a project. It is a thing you now own.

At the end of that year the organisation possesses something it did not possess before: a written, current, authoritative statement of what it will and will not do, in a form that can be enforced without asking anyone.

Not a policy binder. A binder describes intentions. This is a set of decisions that were actually made — by named people, on dates, with the disagreements resolved — about questions the organisation had been avoiding, in some cases for decades.

It has three properties that project cost does not.

It persists. Liz's mapping rule does not expire when the programme ends. It runs every day, unattended, until somebody changes it.

It compounds. The second rule is cheaper than the first because the authority record already exists. The tenth is cheaper still. By the fiftieth, most of the hard institutional questions have been settled once and the remaining work is genuinely translation.

It transfers. It is not tied to the gate enforcing it — Chapter 5 established that the six questions are what any enforcement mechanism needs, so a corpus expressed against them is expressible anywhere. Change gates and the corpus moves.

A note on the balance sheet

Somebody will ask whether this is capitalisable. The honest answer is: probably not, and the accounting treatment is a question for your CFO rather than this paper.

Internally generated intangibles are largely expensed as incurred under both major accounting regimes. A compiled governance corpus will almost certainly go through the P&L as it is built.

Which produces an awkward result worth naming plainly. **The most valuable durable thing the programme produces will not appear anywhere in the financial statements.** It has real value — it transfers in an acquisition, it shows up in diligence, a buyer pays for it, and its absence is a discount — but there is no line for it.

That is precisely why it gets underfunded. The cost is visible and immediate. The asset is invisible and deferred.

Where the return actually comes from

Four places, and only one of them is the obvious one.

Cost per decision. The obvious one. Work that consumed human time now consumes almost none. Real, easy to measure, and the least interesting item on the list.

Speed to launch. This is the one that gets missed. A business that can compile a new product's rules in three weeks can ship products a business that takes nine months cannot ship at all. The corpus is not just a cost saving — it is the thing that determines what the organisation is capable of putting into market. And every rule already compiled makes the next product faster, because the authority record, the failure behaviour, and half the standards are already settled.

Insurability and defensibility. Chapter 2's three parties are asking one question, and the corpus is the only artifact that answers it. What the organisation could not do, why, and on whose authority. An insurer can price that. A regulator can examine it. A plaintiff has to argue against a record rather than against silence.

And the thing nobody budgets for: the organisation now knows its own rules. The programme forced answers to questions that had been unresolved for years — who signs, what the threshold is, what happens when the system is down. Those answers are useful everywhere, not just at the gate. The COO who discovered her benefit system had been down four hours a year did not find that out from a monitoring tool. She found it out because somebody asked question six.

The honest ROI conversation

Do not promise a payback period. You cannot calculate one credibly and the attempt will be picked apart.

What you can say is specific and defensible:

This much of our agent-permitted governance is executable, up from nothing. Here is what remains and what blocks it. The cost per decision on the compiled surface is effectively zero and will not rise. Every future product touching this surface compiles faster than the first one did, because the authority questions are answered. And we can now answer, for any single decision our systems made, which rule applied and who set it — which we could not do last year, and which our insurer, our regulator, and eventually our counterparties are going to ask for.

That is not a payback calculation. It is a description of a capability the organisation did not have, expressed in things that can be measured.

What destroys it

An asset that compounds can also decay, and this one has a specific vulnerability.

Every rule in the corpus passed the six questions. That is what makes it enforceable, and what makes it portable, and what makes the next rule cheaper.

Nothing prevents somebody adding a rule that did not.

The corpus does not degrade through disuse. It degrades through accumulation — through additions made by people who were not there for the original work, do not know why the six questions exist, and have a perfectly reasonable requirement they would like the gate to enforce by Friday.

That is the subject of the next chapter, and it will happen to every organisation that does this.

Chapter 13 — The New Compliance Officer

Eighteen months in, the corpus is running. Two hundred and forty rules, most of the claims surface compiled, the gate doing its work at a cost nobody thinks about any more.

Then Dana starts.

She is the new Head of Compliance. She is good — that is why she was hired. She has spent three weeks reading and she has arrived at the meeting with a list.

Her first item is entirely reasonable.

> *We should be flagging any claim that looks unusual.*

Everyone in the room agrees. Of course you should. It is the sort of thing a compliance function exists to want.

And it cannot be added to the corpus, for reasons that will take the rest of this chapter to explain to her — and which, once she understands them, will make her one of the more effective people in the building.

The three ways a new constraint fails

Dana's request fails at question two. *Unusual* is not testable. It needs a person to say what it means, and until one does there is no rule — only an intention.

She is not being sloppy. She is writing the way compliance has always written, and the way it worked: broad enough to catch things nobody anticipated, resolved by a human at the point of use. That is the interpretive arrangement, and it was correct for four thousand years.

There are two other ways her list will fail, and both are subtler.

Her second item: *check every claim against the sanctions feed before payment.*

That one is testable. It is also unbounded — the feed is a third party with no guaranteed response time, and a rule requiring an answer that may not arrive cannot be enforced at a boundary. It fails question four.

This one is dangerous precisely because it looks fine on the page. Nothing about the sentence signals a problem. It surfaces only when somebody asks what happens when the feed does not respond, and the answer is that the gate hangs, or times out into a behaviour nobody chose.

Her third item: *flag high-value claims for review but let them proceed.*

Testable, bounded, and fails question five. It produces no binding output. If the claim proceeds either way, no rule was enforced — a note was written. Worse, it reintroduces exactly the thing the whole arrangement removed: a judgment to be made later, by a person who will be looking at a claim that already paid.

Three requests. Three different failures. All three reasonable if a human were reading them.

What happens if you just add them anyway

This is the part organisations get wrong, and it is worth being concrete about the mechanism, because "it degrades the corpus" is not a sentence that stops anybody.

Add the untestable one and somebody has to implement it. That somebody will be a developer, and the developer will choose what *unusual* means — probably a threshold, probably reasonable, probably never discussed with anyone. The organisation now has a rule in production that was set by a person with no standing to set it, and nobody knows it happened. Question three has been failed silently.

Add the unbounded one and the gate acquires a dependency it cannot guarantee. Most days it is fine. Then the feed has a bad afternoon, and the failure behaviour that fires is whichever one the timeout handler implements — which nobody chose, which nobody reviewed, and which will apply to every claim in the queue.

Add the non-binding one and you have built a queue. Claims flagged and paid, accumulating in a list somebody is meant to review, that nobody has time to review, that becomes a discoverable record of things the organisation noticed and did nothing about. That is worse than not flagging them.

None of these breaks the gate. That is the trouble. The gate keeps running. The corpus quietly stops being a set of decisions made by people with authority and becomes a mixture of those and defaults chosen by whoever was implementing that week.

The corpus does not decay through neglect. It decays through accumulation.

What Dana's requests become

Nothing on her list gets rejected. All three get converted, and the conversion is the Constraint Architect's job.

Unusual goes back to Dana with a question: what specifically would you like flagged? She comes back with four things — claims from providers with a submission spike, claims where the treatment does not match the diagnosis category, claims above a threshold from a provider under twelve months old, duplicate submissions inside a window. Each of those is testable. Two of them she was not consciously thinking of and only found by being asked.

That conversation takes an afternoon and produces a better control than her original sentence would have, because *unusual* was doing four different jobs and nobody had noticed.

The sanctions check becomes: check against the local cached copy, which returns in bounded time, and route to a human when the cache is older than the agreed window. Dana gets her check. The gate gets a guarantee. And somebody now owns the freshness of the cache, which is a question that had never been asked.

Flag but proceed gets the harder conversation. Do you want this stopped or not? Dana thinks about it and says stopped, above a higher threshold than she first suggested — because when the choice was *flag it and carry on* she picked a low number, and when the choice was *stop it* she wanted to be sure. That is the question forcing a better answer than the original request contained.

The role this creates

None of that happens by itself. Somebody has to sit between Dana and the gate, and that somebody needs three things: to know the six questions cold, to have enough standing that "no, not in that form" carries weight, and to be constitutionally capable of asking the same question a fourth time when the room wants to move on.

This is the Constraint Architect, and this chapter is the reason the role is permanent rather than a project appointment.

The original compilation programme ends. The corpus does not. Rules get added forever — by new compliance officers, by new products, by regulators, by incidents. Every one of those additions has to pass the same six questions the original two hundred and forty passed, or the corpus stops being what it was.

An organisation that funds the programme and not the function has bought an asset with no maintenance.

Dana, six months later

She is the one who catches it now. Somebody from the product team wants a rule that says *material* changes to a policy require re-underwriting, and Dana asks what material means before it gets anywhere near the Constraint Architect. She has learned the six questions, not because anyone made her, but because she got tired of having her requests handed back.

She is also, by some distance, the most effective compliance officer the organisation has had — because her controls now actually operate, every time, on every claim, rather than existing as sentences in a document that people mostly follow.

That is the trade for her profession, and it is the interlude's argument arriving in one person. She gave up writing rules that stretch to cover anything. She got rules that are actually enforced.

Most compliance officers, offered that trade honestly, take it.

The next chapter turns from the organisations doing this work to the companies selling into it, and asks whether the enforcement vendors currently raising money have located their business in the right place.

Chapter 14 — Where the Value Actually Sits

There is a category forming, and it is easy to see. Companies building gates.

They are technically serious. They have solved real problems — evaluation in bounded time, non-bypassable enforcement, tamper-evident records of what was decided. Some are hardware, some are software, some are embedded in orchestration platforms. Where such systems exist, they tend toward the same shape, and Chapter 5 explains why: binary output, bounded evaluation, authority checked separately from correctness, constraints bound before the agent runs, declared behaviour on failure. This is not a survey finding. It is what the situation permits. A gate facing an irreversible act with nobody watching has a narrow set of options, and independent teams working the problem carefully will land inside it.

That convergence is usually read as validation — proof the category is real. It is proof the category is real. It is also the warning.

Convergent architecture commoditises

When independent teams reach the same design, it is because the problem admits one answer. Chapter 5 explained why: the six requirements fall out of the situation, not out of anybody's cleverness.

Designs that fall out of the situation do not stay differentiated. They get implemented repeatedly, described in papers, standardised, and eventually shipped inside something larger as a feature nobody pays separately for. The remaining competition is on latency, form factor, and price — which is a components business, with components margins.

This has happened before, reliably. Firewalls were a category and became a feature. Load balancers, a category, then a checkbox. TLS termination. Container orchestration. Identity providers survived, but the ones that survived did not survive on the protocol.

There is no reason to think enforcement gates escape this. They are more architecturally constrained than any of those, which makes convergence faster rather than slower.

A vendor whose defensibility rests on having built a good gate has built something that will be commoditised by people who read their paper.

The thing that does not converge

Meanwhile, look at what the customer accumulated.

The insurer's two hundred and forty rules are not a copy of anyone else's two hundred and forty rules. They encode Liz's twenty-two years, the argument between medical coding and claims operations, the COO's decision about what happens when the table is down, and four thousand small settlements of questions that only exist in that company.

Every one of those was expensive. None of them is transferable to another insurer. And they compound — the second rule cost less than the first, and the two hundred and fortieth cost a fraction of either.

That is the opposite of convergent. It is specific, accumulated, costly to reproduce, and it grows more valuable the longer it exists.

Whoever holds the corpus holds the customer. Not because they have locked anything, but because the corpus is where the switching cost genuinely lives, and it lives there for reasons that have nothing to do with contracts.

The strategic error, stated plainly

The gate is where the technology is. The corpus is where the value is.

A company positioned entirely at the gate is a component supplier to a business whose real asset it does not touch. It will be selected on latency and price, replaced when something faster appears, and will have no view of why the customer stays or leaves — because the reason is upstream, in a room it was never in.

This is not an argument against building gates. Somebody has to, the work is hard, and the whole arrangement fails without it. It is an argument about where a business is defensible, and those are different questions.

Nobody named here is doing anything wrong. Most of them arrived at enforcement because enforcement was the visible problem — the thing that obviously did not exist, the thing a technical team could see and solve. Compilation looked like the customer's problem, or a professional services problem, or something that would sort itself out.

It is the whole business.

The error the next wave will make

There is a second mistake coming and it is worth naming now, because it is already being made.

The obvious move for a compilation platform is extraction. Point a language model at the corpus, pull out the rules, generate the artifacts. The technology is good enough to do this convincingly, the demos are excellent, and the funding is available.

Chapter 3 explained why this fails, and it is worth repeating in commercial terms.

Extraction assumes the rule exists in the document and needs finding. Frequently it does not. *Who is allowed to approve this* has no answer in the corpus because nobody ever wrote one down — the question was resolved forty times a week by somebody picking up a phone. Asking a model to extract that produces a confident guess, and a confident guess about authority is worse than an admission of ignorance, because it looks like an answer.

Running the extraction repeatedly and taking consensus does not fix it. Sampling resolves variance, not ambiguity. Ten passes over *excessive risk* produce a distribution, and the distribution measures how ambiguous the term is. Averaging it yields a number nobody chose and nobody will defend.

There is a legitimate use for repeated passes, and it is the inverse: if five runs disagree, that policy needs a human. Disagreement as a triage signal is cheap and sound. Disagreement resolved by voting is a fabricated answer with a confidence score attached.

Three prior waves treated the bottleneck as a tooling problem. This wave has by far the best tooling and will make the same error in its most convincing form yet.

Where a company should actually stand

Three positions, and they are not equally good.

The gate. Real, necessary, converging. Build it if you can build it better than everyone else and you are content with a components business. Do not mistake it for a moat.

The corpus. The system of record for compiled governance — what the rules are, who set them, on what authority, superseding what, and whether each one still passes the six questions. This is where the accumulation happens and where the customer's switching cost lives. It is also where nobody is currently standing.

The discovery work. Getting Liz in a room and keeping her there. This does not scale, cannot be productised, and is exactly why it stays valuable — it is a services business, and it is the larger of the two by revenue.

The uncomfortable answer for a founder is that the defensible position requires the services motion to feed the platform, which is a harder company to build and a worse story for investors than a pure software play.

It is also the one that works, for the same reason it worked in ERP: the software captured the configuration, the services captured the decisions, and the firms that did both owned the customer for twenty years.

What to do if you have already built the gate

You are not badly positioned. You are incompletely positioned, and the fix is available.

Move upstream toward the corpus rather than downstream toward more enforcement features. The customer's compiled rules already pass through you — you are the only party that sees every one of them and knows which fail. That is the beginning of the system of record, and nobody else has the vantage point.

Do not add requirements beyond the six. Every proprietary field you introduce is a hook that makes the customer's corpus less portable, which feels like retention and reads, to a sophisticated buyer, as a reason not to sign. The strongest position is a gate that needs exactly the six and nothing more — which will look like the least impressive product in the bake-off, and is the only one whose customer is choosing to stay.

One more thing, and later chapters lean on it heavily.

The six are universal. Not universal because everyone agreed on them, or because a standards body published them, or because they have been widely adopted. Universal because they were derived rather than chosen. They fall out of the situation — an irreversible act, nobody watching — and any gate facing that situation needs all six, whatever it is built from and whoever built it. A gate missing one of them is not a different design philosophy. It is a gate with a hole in it.

That has a consequence worth stating now. A rule written against the six is written against what every gate requires. It is not written for a vendor. Which means the corpus an organisation builds does not belong to whatever is currently enforcing it — it can move, because the thing it was written against is the problem rather than the product.

The next chapter turns from the vendors to the market as a whole — what the shape of this industry is, who makes money in it, and what it will most closely resemble.

Chapter 15 — What This Market Looks Like

There are three businesses here, and they are not the same business.

Most people looking at this see one — a platform that turns policy into executable rules — and they are looking at the smallest of the three.

The three businesses

Discovery. Getting Liz in a room. Running the six questions across a corpus nobody has read end to end in a decade. Finding out that medical coding and claims operations have held incompatible beliefs about authority for eleven years, and then getting somebody senior enough to settle it. Interviewing people about what they actually do when the rule does not fit.

This does not scale. It cannot be productised. It is billed by the day and it is the largest of the three by revenue, for the same reason ERP implementation always dwarfed the licence.

The platform. The system of record for compiled governance. What the rules are, what they compile to, who set each one and on what authority, what it superseded, whether it still passes the six questions, and what breaks if it changes. Version control, simulation, coverage reporting, target generation.

Software margins, small relative to services, and durable — because the corpus accumulates inside it and the customer's switching cost accumulates with it.

Assurance. Independent verification that a corpus is what it claims to be. Conformance to the six. Conflict detection across rules. Whether the coverage figure being reported to the board is honest.

This one does not exist yet. It will, and it will be created the same way SOC 2 was — by a buyer refusing to accept the seller's own word.

Why the services half stays large

The instinct in every technology market is that services shrink as the product matures. Early implementations are expensive, the tooling improves, the work commoditises, and the ratio flips.

That does not happen here, and the reason is Chapter 4's exception layer.

The expensive part of compilation is not extracting rules from documents. It is producing answers that do not exist in any document — deciding what *excessive* means,

settling who is allowed to approve, choosing what happens when the system is down. Those are institutional decisions, made by people with standing, in rooms, at the speed of human disagreement.

No tool does that. A tool can find the ambiguity, route it, track it, and tell you whose desk it is on. It cannot make Legal and Finance agree.

Which means the tooling improving does not shrink the services half. It moves the services half onto exactly the work that needs a person, and away from the work that never needed one. The bill does not fall. It gets spent better.

The ERP comparison, and where it breaks

The obvious analogue is ERP, and it holds for a while.

SAP existed and Accenture built a practice on it, and both were correct simultaneously, because the software captured configuration while the services captured decisions — how this company actually runs its supply chain, what its approval hierarchy really is, which of its processes are load-bearing and which are habit. Firms that did both owned the customer for twenty years.

The parallel is close. Same division of labour, same reason the services survive the tooling improving, same accumulation of customer-specific knowledge inside a vendor relationship.

It breaks in one place, and the break is important.

An ERP implementation finishes. Badly, late, over budget — but it finishes, and afterwards there is a maintenance relationship rather than an implementation one.

Compilation does not finish. Chapter 1 established why, and Chapter 9 made it worse: the better the compilation function gets, the more products the business ships, and the more governance it generates. The engagement does not taper into support. It becomes a standing function that either sits inside the customer, or sits inside a firm the customer retains permanently.

That is a materially better business than ERP implementation was, and it is why the professional services firms will arrive early and stay.

Who actually shows up

The large consultancies will be first and will be well suited to it. This is their existing competence — inventory, workshops, authority mapping, stakeholder wrangling, getting decisions out of executives who would rather not decide. They already sit in the room where governance is discussed. They will build a methodology practice around the six questions within a year of anyone publishing them.

Their weakness is that they will want to leave. Their model is engagement-shaped, and this work does not end. The ones that adapt will build managed-service compilation functions rather than programmes.

The specialist firms will win on depth. Compilation in insurance is not compilation in energy — the domain knowledge is most of the value, and a firm that has compiled claims governance for five insurers will do the sixth in a third of the time. Vertical specialisation is where the margin is, and it is defensible in a way general methodology is not.

The platform companies will try to do all of it and will mostly fail at the services half, because the economics of a software company are hostile to a business that bills people by the day. The ones that succeed will partner rather than hire, and will have to accept that the partner is closer to the customer than they are.

The gate vendors are covered in the last chapter. Some will move upstream. Most will not, and will be absorbed.

And the audit firms have the most interesting position, largely by accident. Assurance over a compiled corpus is close to what they already do — testing whether a control operates as described — and their independence requirement means they cannot also implement it. That separation is exactly what makes assurance credible. They will end up defining what a conformant corpus looks like, and everyone else will build to it.

On sizing this

There is no market sizing in this chapter, and the absence is deliberate.

Anyone producing a number for this today is extrapolating from adjacent markets — GRC software, policy management, compliance services — and the extrapolation is unsound, because the thing being sized has not yet had its demand event. Chapter 2 describes three parties who will compel this work and one of them has not yet issued the requirement that turns interest into procurement.

What can be said is the shape.

The buyer is any organisation that has given a machine permission to do something consequential. That set is currently small and growing quickly, and it is not confined to technology companies — the insurer in this paper is not a technology company.

The spend is front-loaded per surface and recurring in aggregate, because the surface count grows. And the largest single line is people's time, which is why the services half dominates and why the market will be larger than the software revenue suggests.

An honest sizing exercise is possible in about two years, once the first mandate lands. Until then, anyone quoting a figure is guessing with a spreadsheet.

The uncomfortable structural fact

The best position in this market is the hardest one to fund.

It requires services revenue to reach the customer, platform revenue to be worth owning, and enough domain depth to do a vertical properly. That is three motions, and investors prefer one.

The alternative — pure software, no services, sell to the enforcement layer — is cleaner, faster to raise on, and positions the company exactly where Chapter 14 says the value is not.

Somebody will build the harder version. It is the one that ends up owning the corpus, which is the one thing here that appreciates.

The next chapter turns to what an engagement actually looks like week by week, and why the sequencing matters more than the tooling.

Chapter 16 — What the Work Actually Looks Like

Most methodologies for this kind of work are fiction. They present a clean sequence of phases with durations attached, and the durations are wrong because they were estimated by someone who has not sat in the room where the arguing happens.

What follows is a sequence rather than a schedule. The order matters. The durations depend entirely on how quickly you can get four specific people into a room, and that is not a methodology question.

Before anything: decide what the machine may attempt

This is the decision that scopes everything, and it is made before any policy is looked at.

Chapter 4 established that you never compile the corpus. You compile the part of it that touches something a machine is allowed to do. So the first act is not an inventory — it is a boundary. *These actions, on these surfaces, within these limits.*

Narrow it aggressively. A first engagement that covers one workflow and forty rules will finish and teach you something. One that covers a division will not finish.

This decision belongs to the business, not to the programme. If the programme is choosing the scope, the business has not engaged yet and everything downstream will stall.

First: find out what is actually there

Not the policy management system. That is the smallest layer and it is the only one anyone has counted.

For the chosen surface, you are looking for: written policy, the regulations that bind it, the contract terms that bind it harder, the delegated authority behind it, the procedures that describe how it actually runs — and the exception practice, which is in nobody's document and is the largest part.

The exception layer only comes out of people. You get it by sitting with the team and asking what they do when the rule does not fit. The answers arrive slowly and sideways, usually as anecdotes, and the useful ones surface when somebody who has been there twelve years says *well, obviously we don't do that when it's a renewal* and everyone in the room nods and nobody had ever written it down.

Budget more time here than seems reasonable. This is where the surprises live and they are all expensive if found later.

Then: run the six questions

Fast, mechanically, across everything in scope. Most policies fail early, and failing early is cheap.

The Anglo-Saxon screen from Chapter 6 does most of this in a first pass — an analyst can flag the judgment terms on day one without knowing anything about the domain.

The output is not a score. It is a list of blockages sorted by which question failed and whose desk it lands on. That list is the actual project plan, and it will not look like the one that was approved.

Then: the elimination, which nobody scheduled

A meaningful part of what you inventoried will turn out to be dead, duplicated, or not a rule at all.

Dead policy consumes analyst time exactly like live policy right up until someone with standing confirms it is dead. Get that confirmation early and in bulk. It is the cheapest work in the engagement and it reduces everything downstream.

The awkward version is policy that is neither dead nor live — the control nobody has operated in four years but nobody will formally retire, because retiring it requires admitting it was not operating. Those need a person with rank, and they need them early.

Then: the workshops, which are the whole engagement

Everything above is preparation. This is where the money goes and where the value is created.

Four people, repeatedly: the Domain Authority who knows the answer, the Constraint Architect who knows what shape the answer has to take, the authority holder who can decide when the room disagrees, and someone to write it down properly.

The work is question by question, rule by rule. It is slow. A productive session settles three or four rules, and one of those will take most of the session because it turned out to contain a question nobody had ever answered.

Two failure modes to watch for.

The first is the room agreeing too easily. When everyone nods at *we'll handle that case by exception*, nothing has been decided — the exception has been deferred to whoever implements it. The Constraint Architect's job is to notice and ask again.

The second is the authority holder not being available. This is the single most common cause of stall. Their diary is the critical path and nobody plans for it, because on paper they are attending occasionally.

Then: compile, which is the easy part

The artifact falls out of the workshop. Twenty lines of structured text per rule, generated largely mechanically from decisions that have already been made.

This is where organisations expect the difficulty to be and it is not. If compilation is hard at this stage, the workshop did not finish — something is still ambiguous and it is being papered over in the artifact.

Then: simulate before you enforce

Run the compiled rules against historical volume without enforcing anything. Watch what would have been allowed, what would have been denied, and what would have been routed to a person.

This finds two things.

Rules that are wrong — which produce denial patterns nobody expected, usually because a condition was drawn tighter than the workshop realised.

And the volume going to humans — which is the number that determines whether the arrangement is operationally viable. If the compiled rules route thirty percent of volume to a team of thirty, the design does not work and it is far cheaper to know that now.

Simulation is skipped more often than any other step, because the rules look obviously correct on paper. They are not obviously correct on paper.

Then: enforce, narrowly

Start on the highest-consequence surface with the lowest volume, not the reverse. The instinct is to prove value on the big number. The better sequence is to prove correctness where a mistake is visible and recoverable.

Everything not yet enforced is explicitly marked as ungoverned. That distinction has to be visible in the record from day one — Chapter 10's argument applies to partial deployment as much as to outages. An organisation that cannot say which surfaces were governed on a given date has lost the main thing it was building.

And then it does not stop

The programme ends. The function does not.

New rules arrive continuously and every one has to pass the same six questions. Somebody watches the volume going to humans and asks why when it moves. Somebody reexamines rules when the business changes underneath them. Somebody sits between the new compliance officer and the gate.

An organisation that funds the programme and not the function has bought an asset with no maintenance, and Chapter 13 describes what happens to it.

How long

Nobody can tell you, and anybody who does is selling something.

What can be said: the compilation is fast, the discovery is slow, and the workshops are governed entirely by the availability of a small number of senior people who have other jobs. A first surface with forty rules is a matter of weeks if those people are available and a matter of quarters if they are not.

The variance is not in the tooling. It is whether the organisation has decided this matters enough to take four senior people off their day jobs and put them in a room.

That decision, more than any other, determines whether this works.

The next chapter takes the objections seriously, including the two that would sink this if they hold.

Chapter 17 — The Objections

Six of these are serious. Two of them I cannot fully answer.

"This is policy-as-code with a new vocabulary."

The closest objection to correct, and it deserves a careful answer rather than a defensive one.

Policy-as-code works. Rego, Cedar, and the rest compile policy into enforcement that runs at the point of decision, and they do it well. Nothing here improves on that machinery.

What is different is the scope and the bottleneck. Policy-as-code operates on service-boundary authorisation — a domain where the rules were already precise, because they were written by engineers for machines, about resources and identities that already existed in structured form. Compilation there is genuinely a translation problem, and the tooling solves it.

Institutional governance is not that. The rule was written for a person, the authority behind it was never documented, and the answer frequently does not exist until somebody with standing creates it. That is not a translation problem with a bigger corpus. It is a different problem that produces an artifact of the same shape.

If the objection reduces to *the output looks like Rego*, then yes. The output was never the contribution.

"The compiled fraction is too small to matter."

If only a fifth of a corpus compiles, why bother? Because the fraction is not the point — the volume behind it is. The rules that compile are the ones covering high-frequency, repetitive decisions, and those account for the overwhelming majority of actions. Chapter 9's insurer compiled a minority of its governance and covered most of its claims.

The uncompiled remainder is where the hard cases are, and Chapter 9 argued at length that this is where the risk concentrates. That is a reason to staff and watch The Uncompiled Region properly, not a reason to leave the compilable part ungoverned.

"Better models will solve the extraction problem."

They will, and it will not help as much as expected.

Extraction was never the bottleneck. When somebody is asked what *excessive* means, the request usually produces a decision rather than a recollection — because operating practice never forced anyone to settle it. A model can find the ambiguity and cannot resolve it, because resolution is an act of authority rather than an act of interpretation.

The strongest version of this objection is subtler: a model could propose the answer, and a human could approve it, which is much faster than a workshop. That is probably right, and it is a real efficiency. But approval requires someone with standing to actually read and own it, and the moment approval becomes rubber-stamping you have a rule set by a model with a human name attached — which is worse than no rule, because it looks accountable and is not.

"Portability will never happen."

I think this one is partly right, and Chapter 12 concedes the mechanism.

Enforcement vendors have a direct commercial interest in preventing portability. They will support import enthusiastically and make export lossy in ways that are hard to prove. Left to the market, the neutral artifact does not emerge.

The chapter's answer is that portability arrives by mandate rather than agreement — from an insurer who cannot price what it cannot read, or a regulator examining consistency across heterogeneous systems.

That is a prediction, not evidence. It could be wrong. If no mandate materialises, compilation remains a feature of each enforcement platform, the corpus stays captive, and the industry argument in Chapters 14 and 15 substantially weakens.

I do not have a way to resolve this in advance. It is the paper's largest open exposure and it should be read as such.

"A well-drafted policy does not need any of this."

The most interesting objection, because it fails in an instructive way.

Consider a rule that looks fully compiled: *approve automatically when the model's confidence exceeds ninety percent and no outliers are flagged.* Testable. Bounded. Binary. Authority named. It passes every question.

And it contains an assumption nobody wrote down: that a confidence score means the same thing next year as it does today. If the model drifts and begins producing confident errors, the rule continues to be satisfied. The condition holds. The extraction is wrong. Nothing in the rule can detect this, because the rule was drafted assuming permanent model stability and that assumption was never made explicit.

Good drafting does not save you, because the failure is in what the drafter did not know they were assuming. The response is not better drafting — it is a further condition making the assumption explicit and checkable: a current calibration attestation, absent which the rule routes to review regardless of the score.

Which is itself a decision somebody with authority has to make. The objection collapses back into the paper's argument.

"The profession will never form."

Reasonable, and the historical record supports it. The business rules analyst was named, given a manifesto, and absorbed back into business analysis within a decade.

Chapter 3's answer is that four conditions have now changed simultaneously — no interval for review, a fixed compilation target, external parties compelling the work, and a corpus that never empties. The last one matters most: the earlier role dissolved partly because the work looked finite, and roles built around finite work do not survive its completion.

But this is an argument about why it should form, not evidence that it will. Professions form for reasons that include credentialing bodies, career paths, and salary premiums, none of which exist here yet.

Ask again in five years. If there is no qualification, no job title in common use, and no identifiable career path, this objection was right.

"You are describing your own consulting engagement."

Fair to ask, and worth answering plainly.

The argument is structured to be checkable independently of who made it. The six questions are derived rather than asserted, and Chapter 5 invites the reader to find a seventh. The funnel figures are marked as illustrative because no measurement exists. The claims environment is a constructed composite and says so. The largest evidence gap — nobody has published corpus attrition — is stated rather than filled with an estimate.

The parts most likely to be self-serving are the commercial chapters, and they are the ones a sceptical reader should discount most heavily. The structural argument in Part I stands or falls on its own.

What would actually falsify this

Four things, and they are not rhetorical.

An institution of real size that compiled its full governance corpus and held that state through a normal year. Chapter 1's central claim predicts none exists.

A seventh load-bearing requirement. Not logging, notification, or escalation, all of which are covered. If one exists, Chapter 5's derivation is incomplete and everything built on it needs revisiting.

Corpus attrition data showing the funnel does not narrow. If real assessments find that most institutional policy compiles cleanly, then Chapters 7 and 8 have overstated the problem badly.

And a compilation platform that resolves ambiguity without human authority, defensibly. If a system can decide what *excessive* means in a way that survives a regulator asking who set it, the services argument collapses and this is a software market after all.

None of those has been tested, because the assessment is new. All four are testable by anyone willing to do the work.

The final chapter sets out what remains unresolved, and what the next paper in this series has to address.

Chapter 18 — What Is Still Missing

Six problems this paper does not solve. Four of them are open research; two are things somebody has to build.

Nobody has measured the funnel

The most conspicuous gap. Chapter 7's figures are illustrative and the chapter says so, but that admission does not repair the hole.

What is needed is unglamorous: one organisation of real size, one defined surface, the six questions run across everything in scope, and the attrition published — how many statements entered, how many fell at each question, and what the resolution of each blockage actually cost in senior time.

Nobody has done this because the assessment is new and the organisations doing the work have commercial reasons to keep the result. The first published attrition figures will be the most valuable number in the category, and they will probably come from a regulator requiring disclosure rather than from a firm volunteering it.

There is no way to verify a compiled rule means what the policy meant

This is the deepest problem in the paper and it is genuinely unsolved.

A compiled rule can be verified against itself — it is well-formed, all six resolve, it terminates, it produces binary output. None of that establishes that it captures what the organisation actually intended.

Chapter 17's confidence-threshold example shows the failure clearly. A rule can be perfectly compiled, perfectly enforced, perfectly evidenced, and wrong, because the drafter encoded an assumption they did not know they held. The gate proves the system stayed inside declared bounds. It says nothing about whether the bounds were correctly declared.

Simulation helps and does not close it. Simulation tests against history, and the cases that matter are the ones history does not contain.

I do not know how this gets solved. Formal methods offer something for narrow domains. Adversarial review — a second Constraint Architect trying to break the rule — is probably the practical answer for now, and it is a practice rather than a technique.

Cross-jurisdictional conflict has no resolution procedure

An organisation operating in four countries will find that some rules are incompatible with others, and that in the interpretive arrangement this was survivable because a person resolved each case quietly.

Compilation forces the conflict into the open, which is progress. It does not say what to do about it. Compile the strictest and you may violate a permission you are commercially obliged to grant. Compile per jurisdiction and you need a jurisdiction determination at runtime, which is another rule with its own six questions and its own ambiguity.

There is no general answer here, and the tooling implication is significant: a corpus is not a flat set of rules but something with scope and precedence, and precedence has to be declared by someone with authority to declare it. That machinery does not exist in any current platform.

Nobody knows when a rule has stopped fitting reality

A compiled rule is a decision frozen at a moment. The business changes underneath it continuously.

Liz's mapping was correct when she made it. Three years later the procedure has changed, the codes have been revised, and the rule is still running — correctly, deterministically, on an assumption that expired.

Detecting this is not a matter of monitoring the rule, because the rule is behaving exactly as specified. It requires noticing that the world has moved. The exception queue is a partial signal — Chapter 9's dashboard — but it only catches drift that produces visible routing changes, and the dangerous drift is the kind that keeps allowing things it should now be stopping.

Every rule probably needs an expiry or a review trigger. Nobody has worked out what triggers it.

There is no assurance standard, and no one qualified to attest

Chapter 15 predicted an assurance business. It does not exist, and neither does the thing it would attest to.

What would a conformance opinion actually say? That every rule passes the six? That the coverage figure is honest? That the authority records supporting it are current? Each of those is testable in principle and none is defined in practice.

Nor is there anyone qualified. Audit firms have the independence and the methodology but not the competence yet. Constraint Architects have the competence and cannot attest to work adjacent to their own.

This will get built, and the shape it takes will determine a great deal — because whoever defines a conformant corpus effectively defines the compilation target for everyone.

And the compilation record has no standard either

Chapter 12 argued that a portable corpus must carry its provenance: who authored each condition, who approved it, on what authority, superseding what.

The requirement is clear. The format is not, and no platform currently produces one.

This matters more than it sounds. The enforcement gets witnessed at the moment of decision — that argument is made properly in the prior paper in this series. But the accountability lives upstream, in the rule, set by a person on a date. A corpus that moves without that record is enforcement without accountability, and the question a plaintiff asks is not *was the rule applied* but *who set it and were they allowed to*.

Witnessing the compilation is the obvious extension. Nobody has specified it.

What the next paper has to do

The series has established that governance migrates to the execution boundary, that a minimal constitutional structure emerges there, that an independent evidence plane is structurally required, and now that translation into that structure becomes a permanent institutional function.

The unresolved thread running through all four is economic. Chapter 12 argued the corpus is an asset with a real return and no place on the balance sheet. Chapter 15 argued the market shape without sizing it. Chapter 9's Jevons argument says the volume of governed decisions rises without bound as adjudication cost falls.

None of that has been costed. What does a governed decision actually cost, who pays for the evidence, what does an insurer price against, and what happens to those economics when the volume rises by three orders of magnitude — these are open, and they determine whether any of this is affordable at the scale it is heading toward.

That is the next paper.

The one-sentence version

For four thousand years we wrote rules that a person would interpret afterwards, and built our institutions and our professions around the interpreting.

There is no afterwards now. The interpretation has to happen first, in a room, before the case exists — which is harder, and slower, and requires exactly the people every previous wave of automation let go.

It never finishes. That is not the problem with it. It is what makes it a profession.

End of manuscript — Version 2.0

Upstream: *Where Human-in-the-Loop Goes at Machine Speed*

18 chapters. 1 interlude.

Jonathan M. Watson
Independent Researcher
jonmwatson@gmail.com
August 01, 2026

Glossary of Terms

Terms are defined as used in this paper. Where a term is also used elsewhere with a different meaning, that usage is not implied here.

Agent — Software that takes consequential action without a person present at the moment of action. Distinguished throughout from software that recommends, prepares, or routes work to a human decision-maker.

Anglo-Saxon screen — A ten-minute triage described in Chapter 6: highlight every word in a policy that entered English from Norman French or Latin. Those terms carry most of the ambiguity, because they arrived to describe delegated authority exercised at a distance and were deliberately broad enough to cover cases nobody could enumerate. A rule of thumb with a high hit rate, not a law.

Assurance — Independent verification that a compiled corpus is what it claims to be: conformant to the six questions, internally consistent, and honestly reported. Predicted in Chapter 15 as a third business alongside discovery and platform. Does not yet exist.

Authority holder — The person who can decide when the room disagrees. Legal for who signs; risk for what a threshold means; the executive who carries the consequence for what happens when governance is unavailable. Not present full-time. Their availability is the most common cause of programme stall.

Binding output — The fifth question. The result of a rule's evaluation must determine what happens next. If the system can proceed regardless of the answer, no rule was enforced — a suggestion was logged.

Compilation — The conversion of governance written for a person to interpret into a form a machine can apply unaided. A rule is compiled when all six questions are answered; it is not compiled when any one is not. There is no partial credit. Distinguished sharply from the gate's decision at runtime.

Compiled artifact — The vendor-neutral expression of a compiled rule against the six questions. The durable object. Distinguished from the runtime target, which is a rendering of it in whatever language a particular gate speaks.

Compiled Region — The portion of an organisation's governance that has been compiled and is enforced by machine. Deterministic, unattended, near-zero cost per decision. Grows continuously and never covers the whole corpus.

Constraint Architect — The person who knows what the six questions require and how to extract an answer from a domain expert in a form that will hold. Part analyst, part interviewer, part draftsman. The distinguishing skill is recognising that *we'll handle that case by exception* is not an answer, and asking again. A permanent function rather than a project role.

Corpus — The full body of an organisation's governance: written policy, legislation and regulation, contracts, standards, delegated authority, risk appetite, procedures, and the undocumented exception layer. Larger than the policy management system reports, usually by a wide margin.

Coverage figure — The measure a compilation programme reports: how much of the governance covering agent-permitted actions is executable, what blocks the remainder, and whose desk each blockage sits on. Measurable, moves quarter over quarter, and is the only form of answer to what insurers, regulators, and counterparties are asking.

Declared Failure Sovereignty — The universal structure underlying the sixth question. The direction of failure varies by domain — stop, hand back to a person, enter a safe state, continue in a restricted mode — but five properties are constant: the behaviour was decided in advance, by someone with standing, the machine does not invent authority, authority resolves somewhere specific, and the transition is recorded.

Discovery — The interpretive work of finding out what an organisation's governance actually is, including the parts held only in people's heads. Does not scale, cannot be productised, and is the largest of the three businesses by revenue.

Domain Authority — The person who holds the knowledge and has standing to say what the answer is. Does not need to understand compilation and should not be required to.

Exception layer — The accumulated practice of what people do when the rule does not fit. Never drafted, never approved, never stored, never retired. Argued in Chapter 4 to be the largest part of the governance corpus, on the grounds that written policy is bounded by the cost of writing things down and exception practice is bounded by nothing.

Gate — The mechanism that evaluates a proposed action against compiled rules and returns a binding allow-or-deny determination before the action commits. Operates in milliseconds, unattended, at volume. Convergently designed and therefore, this paper argues, commoditising.

Interpretive arrangement — The four-thousand-year-old default in which ambiguity is permitted at the moment of action and resolved afterwards by a person with standing. Contracts interpreted when disputed, accounts audited after the year closes, courts operating entirely in the past tense. Not a flaw in the design; it is the design.

Judgment term — A word whose meaning requires a person to settle it: *appropriate*, *material*, *reasonable*, *excessive*, *necessary*. Until someone with standing states what it means, there is no rule — only an intention.

Portability — The property of a compiled artifact that allows it to be enforced by a different gate without recompilation. Follows from the universality of the six questions. Predicted in Chapter 12 to arrive by mandate from insurers or regulators rather than by vendor cooperation, because vendors have a direct interest in preventing it.

Provenance — The record of who authored each condition in a compiled rule, who approved it, on what authority, superseding what, and when. Must travel with the artifact. A rule that moves without it is enforcement without accountability.

Six questions — The minimal set a rule must resolve before a machine can enforce it: what exactly is being proposed; what the rule says in terms a machine can test; who is allowed; whether the check finishes in the available window; what comes out and whether it binds; and what happens when the check cannot run. Derived from the situation rather than chosen, and therefore universal — any gate facing an irreversible act with nobody watching requires all six.

Termination — The fourth question. Every lookup a rule requires must return, inside the window, every time. The requirement most often missed, because in human hands every rule terminates — the person simply stops thinking and decides.

Uncompiled Frontier — The boundary between the Compiled Region and the Uncompiled Region. Its position is set not by the ratio of translated to untranslated policy but by the relationship between what machines are permitted to attempt and what has been settled about those attempts. It moves as translation proceeds and as new governance is produced. It does not close.

Uncompiled Region — Everything not compiled: ambiguous, contextual, decided by people at human speed. Permanent, for two reasons — new governance arrives faster than old governance compiles, and some cases are routed to humans deliberately because the organisation chose not to hand that judgment to a machine. Where the hard cases and the litigation concentrate.

Universality (of the six) — The property that makes the compiled artifact independent of any vendor. The six are universal not because anyone agreed on them but because they were derived rather than chosen: they fall out of the situation, and a gate missing one is not a different design philosophy but a gate with a hole in it.

End of Glossary

Bibliography — Version 2.0
Upstream: *How Human Governance Becomes Executable, and Why the Work Never Ends.*

Safety, real-time systems, and operational practice

Buttazzo, G. (2011). Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications, 3rd ed. Springer.

Hollnagel, E. (2014). Safety-I and Safety-II: The Past and Future of Safety Management. Abingdon: Routledge.

International Electrotechnical Commission (2010). IEC 61508: Functional Safety of Electrical/Electronic/Programmable Electronic Safety-related Systems. Geneva.

Kopetz, H. (2011). Real-Time Systems: Design Principles for Distributed Embedded Applications, 2nd ed. Springer.

Insurance and risk transfer

Fenwick (2026). The End of "Silent AI"? Emerging AI Exclusions, Coverage Fragmentation, and Practical Implications for Policyholders. June 2026.

Howden Broking Group (2021). Cyber Insurance: A Hard Reset. London.

Insurance Services Office (Verisk Analytics) (2026). Endorsements CG 40 47, CG 40 48, and CG 35 08, January 2026 edition.

The Insurer (2026). "Standalone AI liability market takes shape with underwriting discipline key to MGA success." 24 April 2026.

The Insurer (2026). "Verisk weighs new exclusions for agentic AI risks." 10 July 2026.

Regulation, standards, and assurance

American Institute of Certified Public Accountants (2017). Trust Services Criteria for Security, Availability, Processing Integrity, Confidentiality, and Privacy, with 2022 revised points of focus. Durham, NC: AICPA.

Committee of Sponsoring Organizations of the Treadway Commission (2017). Enterprise Risk Management — Integrating with Strategy and Performance.

Executive Order 14028 (2021). Improving the Nation's Cybersecurity. 12 May 2021.

Financial Stability Board (2013). Principles for an Effective Risk Appetite Framework.

International Monetary Fund (2026). How Agentic AI Will Reshape Payments. IMF Notes Vol. 2026, Issue 004, April 2026.

National Telecommunications and Information Administration (2021). The Minimum Elements for a Software Bill of Materials. July 2021.

U.S. Securities and Exchange Commission (2010). Risk Management Controls for Brokers or Dealers with Market Access, Release No. 34-63241, File No. S7-16-10; 17 C.F.R. § 240.15c3-5.

Prior art in executable policy

Amazon Web Services. Cedar Policy Language. Open source, Linux Foundation.

Athan, T., Boley, H., Governatori, G., Palmirani, M., Paschke, A., & Wyner, A. (2013). OASIS LegalRuleML. Proceedings of the 14th International Conference on Artificial Intelligence and Law (ICAIL 2013). ACM, 3–12.

Business Rules Group (2003). The Business Rules Manifesto. Ed. R.G. Ross.

Feigenbaum, E.A. (1977). The art of artificial intelligence: themes and case studies of knowledge engineering. Proceedings of the 5th International Joint Conference on Artificial Intelligence, vol. 2, 1014–1029.

HashiCorp. Sentinel Policy as Code Framework.

Hayes-Roth, F., Waterman, D.A., & Lenat, D.B. (eds.) (1983). Building Expert Systems. Addison-Wesley.

Kyverno. Kyverno: Kubernetes Native Policy Management. CNCF graduated project.

Merigoux, D., Chataing, N., & Protzenko, J. (2021). Catala: A Programming Language for the Law. Proceedings of the ACM on Programming Languages, 5(ICFP), Article 77, 1–29.

National Institute of Standards and Technology. Open Security Controls Assessment Language (OSCAL), version 1.1.

OASIS (2018). Akoma Ntoso Version 1.0: XML Vocabulary for Parliamentary, Legislative and Judiciary Documents. OASIS Standard, August 2018.

Object Management Group. Decision Model and Notation (DMN), version 1.5, 2023.

Open Policy Agent. Open Policy Agent and the Rego Policy Language. CNCF graduated project.

Ross, R.G. (2003). Principles of the Business Rule Approach. Addison-Wesley.

von Halle, B. (2001). Business Rules Applied. Wiley.

Language, law, and tacit knowledge

Crystal, D. (2004). The Stories of English. Penguin.

Mellinkoff, D. (1963). The Language of the Law. Little, Brown.

Pleading in English Act 1362 (36 Edw. III c. 15).

Proceedings in Courts of Justice Act 1730 (4 Geo. II c. 26).

Polanyi, M. (1966). The Tacit Dimension. Garden City, NY: Doubleday.

Roth, M.T. (1997). Law Collections from Mesopotamia and Asia Minor, 2nd ed. Writings from the Ancient World, vol. 6. Atlanta: Society of Biblical Literature.

Economics and labour

Autor, D.H. (2015). Why Are There Still So Many Jobs? The History and Future of Workplace Automation. Journal of Economic Perspectives, 29(3), 3–30.

Jevons, W.S. (1865). The Coal Question: An Inquiry Concerning the Progress of the Nation, and the Probable Exhaustion of Our Coal-Mines. Macmillan.

Sector data

Council for Affordable Quality Healthcare (2025). 2024 CAQH Index Report. Washington, DC: CAQH.

Centers for Medicare & Medicaid Services. Transparency in Coverage Public Use File, Plan Year 2024. Analysed by MoneyGeek, January 2026.

Experian Health (2025). 2025 State of Claims Report.

Prior work in this series

Watson, J.M. Constitutional Compression: The Emergence of Minimal Governance at Machine Speed.

End of Bibliography — Version 2.0.